Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Analysis of financial markets using machine learning

Bachelor Thesis

2026
Marek Šugár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Analysis of financial markets using machine learning

Bachelor Thesis

Study Programme:	Data Science
Field of Study:	Computer Science and Mathematics
Department:	Department of Applied Mathematics and Statistics
Supervisor:	Mgr. Samuel Rosa, PhD.

Bratislava, 2026
Marek Šugár

17424822

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Marek Šugár
Študijný program: dátová veda (Medziodborové štúdium, bakalársky I. st., denná forma)
Študijné odbory: informatika
matematika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Analysis of financial markets using machine learning
Analýza finančných trhov pomocou strojového učenia

Anotácia: Modelovanie a predikcie na finančných trhoch sú dlho skúmaným problémom v odbornej literatúre. Tento problém bol historicky riešený špecifickými metódami finančnej matematiky, ale s rozvojom a úspešnými aplikáciami strojového učenia vo všeobecnosti začali byť metódy strojového učenia používané aj na predikcie správania finančných trhov. Cieľom tejto práce je skonštruovať predikčný model pre vybrané akcie, ktorého súčasťou je nájdenie a výber relevantných vysvetľujúcich premenných, ako aj výber správneho modelu. Následne bude možné analyzovať, či sa získané predikcie dajú úspešne použiť na konštrukciu investičného portfólia.

Vedúci: Mgr. Samuel Rosa, PhD.
Katedra: FMFI.KAMŠ - Katedra aplikovanej matematiky a štatistiky
Vedúci katedry: doc. Mgr. Igor Melicherčík, PhD.

Spôsob sprístupnenia elektronickej verzie práce:
bez obmedzenia

Dátum zadania: 02.10.2025

Dátum schválenia: 28.10.2025 doc. Mgr. Tomáš Vinař, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

17424822

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Marek Šugár
Study programme:	Data Science (Joint degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science Mathematics
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Analysis of financial markets using machine learning

Annotation: Modeling and prediction of stocks in financial markets have long been a researched problem. Historically, this problem was approached using specific methods of financial mathematics, but with the significant development and success of machine learning in general, machine learning methods have also begun to be applied to predicting the behavior of financial markets. The aim of this thesis is to create a prediction model for selected stocks, which involves the appropriate selection and identification of relevant explanatory variables, as well as the suitable choice of model. Subsequently, it will be possible to analyze whether the obtained predictions can be used for successful creation of an investment portfolio.

Supervisor:	Mgr. Samuel Rosa, PhD.
Department:	FMFI.KAMŠ - Department of Applied Mathematics and Statistics
Head of department:	doc. Mgr. Igor Melicherčík, PhD.
Assigned:	02.10.2025
Approved:	28.10.2025

doc. Mgr. Tomáš Vinař, PhD.
Guarantor of Study Programme

.....
Student

.....
Supervisor

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled "Analysis of financial markets using machine learning", including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the LLM ChatGPT only for the keyword-based searches in particular Python libraries' documentations and thesis' grammar checks. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: This way I would like to thank my supervisor Dr. Rosa, for his mentorship, spent time and intriguing feedback throughout the whole process of thesis' creation. Last but not least, I would like to express thanks to all my family and friends for their endless support and motivation.

Abstract

ŠUGÁR, Marek: Analysis of financial markets using machine learning [Bachelor Thesis], Comenius University in Bratislava, Faculty of Mathematics, Physics and Informatics, Department of Applied Mathematics and Statistics; Supervisor: Mgr. Samuel Rosa, PhD., Bratislava, 2026,

The goal of the presented bachelor thesis is to evaluate the predictability of the Information Technology (IT) segment of stocks traded on the US stock markets between September 2017 and January 2026. Using a unified data-pipeline of selected features, we employ particular regression machine learning algorithms to predict the next-day closing price of individual stocks. The used frameworks are Linear regression, Lasso and Ridge regularized regressions, k-Nearest Neighbours, Random Forest Regression, and the ARIMA model. Our results indicate the fairly difficult predictability of the vast majority of stocks in IT segment. Furthermore, we employ these models in portfolio optimization based on the Markowitz mean-variance model. The study empirically demonstrates that obtained predictions have relatively limited impact on portfolio performance in comparison with market benchmarks.

Keywords: quantitative finance, machine learning, stock markets, predictive analysis

Abstrakt

ŠUGÁR, Marek: Analýza finančných trhov pomocou strojového učenia [Bakalárska práca], Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky, Katedra aplikovanej matematiky a štatistiky; Školiteľ: Mgr. Samuel Rosa, PhD., Bratislava, 2026,

Cieľom predkladanej bakalárskej práce je zhodnotiť predikovateľnosť akciového segmentu Informačné Technológie (IT) na akciových burzách v USA v rozmedzí septembra 2017 až januára 2026. Využitím jednotnej sady prediktorov boli implementované vybrané regresné algoritmy strojového učenia, uspôsobené na predikovanie uzatváracej ceny akcie v nasledujúci deň. Využité algoritmy boli lineárna regresia, Lasso a hrebeňová regresia, k-najbližších susedov, náhodné regresné lesy a ARIMA model. Výsledky naznačujú relatívne nízku predikovateľnosť značnej väčšiny akcií v IT segmente. Ďalej bol implementovaný algoritmus optimalizácie portfólia na základoch Markowitzovho mean-variance modelu, ktorý využíva predikcie modelov. Výsledky práce empiricky demonštrujú relatívne nízky vplyv získaných predikcií na výkonnosť portfólia voči trhovým indexom.

Kľúčové slová: kvantitatívne financie, strojové učenie, akciové trhy, prediktívna analýza

Contents

Introduction

"In this business, it's easy to confuse luck with brains."
— Jim Simons (founder of Renaissance Technologies)

Quantitative research of financial markets and the prediction analysis of assets within them have long been a researched topic and a point of interest with high application potential. Quite a few studies have employed conventional machine learning (ML) frameworks in both regression and classification tasks regarding the price development of certain stocks [29].

In general, price development is a complex and non-linear relationship affected by a large number of external factors [35]. Having the ability to approximately predict the course of stocks in the future is of great importance for both retail and large-scale investors.

Among a number of studies implementing ML frameworks in stock price prediction, many of them tend to focus on a very limited number of stocks [13, 32, 35] or on conventional stock indices [18, 27]. There are not many ML-focused studies focusing on a unified group of stocks related to common market segment, country, etc. What is more, some of these papers tend to focus solely on prediction modeling and do not apply these models in subsequent practical analyses with application potential.

What needs to be clarified is that there are quite a few studies related to portfolio optimization, which apply techniques on a coherent sets of stocks. However, they focus particularly on portfolio optimization models, rather than ML-driven predictions (e.g., [11], [15]).

Subsequently, there are particular studies, which aim to connect ML-driven predictions and portfolio optimization (e.g., [4], [10], [12], [21]). However, very few of these ML-driven studies study a one particular market segment. On top of that, there are very few of such studies, which apply ML frameworks on the most recent market data.

The thesis focuses on a specific stock market segment. Among the 500 largest companies listed on stock exchanges in the USA by market capitalization, a total of 65 belong to the Information Technology (IT) sector. The main aim of the research conducted in the thesis is to explore and evaluate the predictability of the segment in the recent time interval, by applying ML prediction models.

Consequently, the acquired models will be further evaluated through a portfolio optimization framework, built upon the Markowitz mean-variance portfolio model. From a technological perspective, notable result of the thesis is a complex pipeline consisting of data preprocessing of a introduced unified set of predictors, day-to-day closing price prediction of individual stocks and, subsequent application, thus evaluation, in portfolio strategy optimization. The pipeline is modular, meaning it could serve as a prediction and portfolio management engine for any set of financial instruments available.

From a broader perspective, the thesis also presents a study of the predictability of an entire market segment in the US stock market during specific time, where both surges and notable crises occured.

Chapter 1

Machine learning foundations

The aim of this chapter is to overview the methods of machine learning, which are going to be used for the purposes of predictive regression modeling in this thesis. The chapter starts with a brief overview of machine learning as a method, followed by descriptions of the individual algorithms used.

1.1 Machine learning as a method

Nowadays, we can say that machine learning (ML) has become a widely known term in the field of data science. It stands for the subset of artificial intelligence enabled to identify patterns in data [3]. Generally speaking, ML implements algorithms capable of learning from data and generalizing patterns to be able to make predictions on unseen data, which means being able to perform tasks without being explicitly told to [14].

Mathematically speaking, ML builds on the idea of having a quantitative target variable Y and p various predictors $X_1, X_2, \ldots, X_p$. We assume the existence of an underlying function f such that

$$Y = f(X_1, X_2, \ldots, X_p) + \epsilon, \tag{1.1}$$

where ϵ is a random noise independent of $X_1, X_2, \ldots, X_p$. Algorithms and methods of ML aim to estimate the function f by using a so-called **training dataset** of observed data.

Suppose we have n observations of the introduced predictors and the corresponding target Y as $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^{n}$, where $X_{i,q}$ denotes value of the q-th predictor in the i-th observation. Then, ML algorithms use this dataset to approximate function f by function $\hat{f}$, such that $\hat{f}(X_{i,1}, X_{i,2}, \ldots, X_{i,p}) = \hat{Y}_i \approx Y_i$, for all $i = 1, 2, \ldots, n$. With the obtained $\hat{f}$, we can make predictions of Y for new (unseen before) values of predictors $X_1, X_2, \ldots, X_p$.

Based on the methodology used to find $\hat{f}$, we can distinguish two methods – parametric and non-parametric. **Parametric methods** seek to approximate f by specifically optimizing a various number of parameters, which then explicitly define the function $\hat{f}$. This involves a prior assumption about the functional form of f (e.g., linear, polynomial, etc.). Then, using specific algorithms, the parameters of the function are estimated from the training data, resulting in $\hat{f}$. Examples of such methods used in this thesis are linear regression or its regularized variants.

On the other hand, **non-parametric methods** do not assume any specific functional form of f. Instead, they rely on patterns included in the data itself to roughly approximate f. Generalisation to unseen data is achieved by identifying patterns and similarities in the training data, without explicitly defining a parametric class of functions $\hat{f}$. An example of such a method used in this thesis is the k-Nearest neighbours algorithm or the random forest algorithm.

Lastly, based on the character of the Y, we can distinguish regression and classification tasks. In **regression** tasks, Y is a continuous variable, while in **classification** tasks, Y is a categorical variable.

1.2 Bias-variance trade-off

Lastly, in order to provide a complete overview of the theoretical foundations of ML, it is important to mention the concepts of overfitting and underfitting [30]. The key concepts associated with these phenomena are **bias** and **variance**.

Bias is the error arising from incorrect assumptions made by the model. One example is assuming that f belongs to a certain functional class; however, the true underlying function may be much more complex (e.g., restricting f to be linear instead of polynomial). It refers to the error that occurs when the true f is much more complex than the estimated $\hat{f}$, resulting in systematically incorrect assumptions by the model, regardless of the number of training samples.

Variance is the error arising from the model's sensitivity to perturbations in the training dataset. The problem occurs when a particular training set yields a trained model, while another, slightly perturbed dataset, yields a significantly different model, which usually leads to unstable generalization performance. More formally, variance refers to the amount by which $\hat{f}$ would change if the model were built using different training datasets. In the ideal case, the estimation of f should not vary between training sets – the model should generalize rather than memorize.

In practice, the goal is to choose the model complexity (e.g., by assuming a functional type or by selecting a particular model) to achieve lowest possible bias and variance simultaneously (see Figure 1.1).

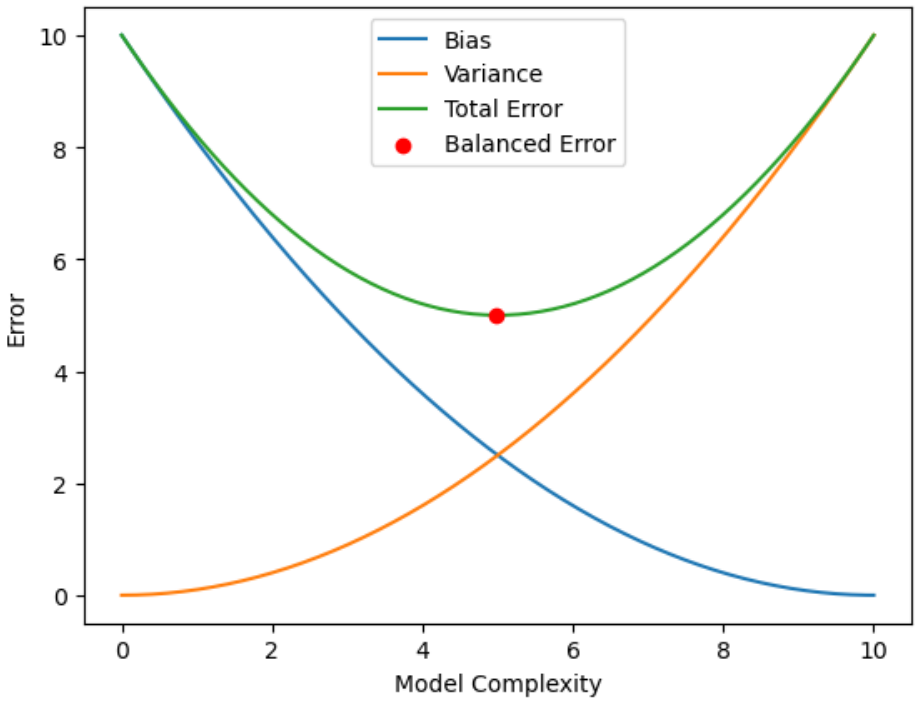

Figure 1.1: Balancing the total error of the model through its bias and variance.

Familiar terms connected to bias and variance are the aforementioned overfitting and underfitting. Typically, in models with low complexity (high bias and low variance), a situation may occur in which the model performs poorly on both training and testing data – this is called underfitting.

On the other hand, the opposite situation arises when the model is overly complex (low bias and high variance). In this case, typically the model performs well on training data; however, it performs significantly worse on the testing data, as it effectively memorizes the training data. This situations is referred to as overfitting.

1.3 Linear regression

Linear regression is one of the most fundamental methods in ML. It is a parametric method used for regression tasks, assuming that the true relationship between the predictors and the target variable is linear [14].

Given a training dataset of n observations $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^{n}$, we can formulate the linear relationship as

$$Y_i = \beta_0 + \beta_1 X_{i,1} + \ldots + \beta_p X_{i,p} + \epsilon_i, \qquad i = 1, 2, \ldots, n, \tag{1.2}$$

with ϵ_i representing an independent noise. Formulating these equations to a more convenient matrix notation, we obtain

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \tag{1.3}$$

where

$$\mathbf{Y} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} 1 & X_{1,1} & \dots & X_{1,p} \\ 1 & X_{2,1} & \dots & X_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & X_{n,1} & \dots & X_{n,p} \end{bmatrix}, \qquad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}, \qquad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}.$$

In order to derive one of the methods to approximate the parameters $\boldsymbol{\beta}$, it is necessary to define **residuals**. A residual represents the deviation of the obtained approximation by the model from the true value of the target (Figure 1.2). When the model, defined by certain approximed $\hat{\boldsymbol{\beta}}$, approximates particular Y as $\hat{Y}$, then $Y - \hat{Y}$ is the residual.

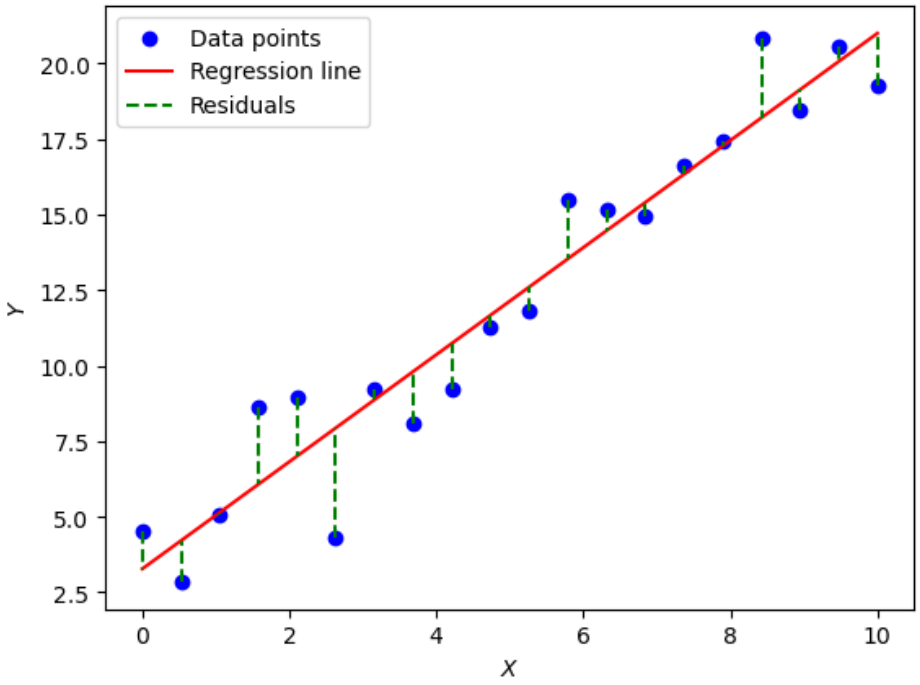

Figure 1.2: Illustrative example of residuals in the context of linear regression.

Using the residuals, we can measure the goodness of fit. We aim to define the linear relation to be as close as possible to the training data. In matrix notation, we can represent the residuals for all n available observations as $\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}$. To ensure that large errors are penalised more heavily and all of the residuals contribute positively, regardless of their sign, the general error function is defined as

$$(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2 \tag{1.4}$$

or in an equivalent way as sum of squared residuals as

$$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2,$$

where $\hat{Y}_i$ denotes the model's approximation of Y_i.

Based on the Equation (1.4), we can derive an optimization approach to derive $\hat{\beta} \approx \beta$ as

$$\arg\min_{\hat{\beta}} \|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2. \tag{1.5}$$

The approach in Equation (1.5) stands for **least-squares estimation**. Other estimation methods for $\hat{\beta}$ exist; however, only the least-squares approach is included in the chapter, as it is used in the subsequent computational implementation.

Apart from the approximation methods, it is possible to derive the closed form of the solution to the Equation (1.5) [30]. Straightforwardly, the value of the derivative with respect to the $\hat{\boldsymbol{\beta}}$ of the Equation (1.4) is set to zero as

$$\frac{\partial}{\partial\hat{\boldsymbol{\beta}}}\|\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2 = -2\mathbf{X}^\top(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = 0$$

and then, solving for $\hat{\boldsymbol{\beta}}$ yields

$$\mathbf{X}^\top\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^\top\mathbf{Y} \Rightarrow \hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{Y}. \tag{1.6}$$

Such solution arises from the convexity in β of the function in Equation (1.4).

1.4 Regularized variants of linear regression

Linear regression might overfit certain types of datasets. This problem arises with large values of the parameters in $\boldsymbol{\beta}$. Relatively large values of the parameters result in significant changes in predictions, even under slight input perturbations, thus making the predictions potentially imprecise. There exist a plenty of methods involving explicit feature selection, resulting in omitting the features with large $\hat{\beta}_i$; however, by using **regularized methods**, the approach of feature selection could be avoided [14, 30].

Continue with Equation (1.5) and let us add a specific term taking into account the sum of absolute values of the single parameters in $\hat{\boldsymbol{\beta}}$

$$\arg\min_{\hat{\boldsymbol{\beta}}} (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top(\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \lambda\|\hat{\boldsymbol{\beta}}\|_1, \tag{1.7}$$

where $\lambda \geq 0$ is a so-called tuning parameter and $\|\hat{\boldsymbol{\beta}}\|_1 = \sum_{i=1}^{p}|\hat{\boldsymbol{\beta}}_{\boldsymbol{i}}|$ is a L1 norm. Equation (1.7) trades off two criteria – on the one hand, equally as in linear regression, the model seeks $\hat{\boldsymbol{\beta}}$ which fits the data the best, but on the other hand, the model seeks a

fit with the smallest possible sum of absolute values of the elements in $\hat{\boldsymbol{\beta}}$. This method is called **Lasso regularization** (Least Absolute Shrinkage and Selection Operator) of linear regression and fully aligns with the objectives of explicitly preserving all the features, while limiting the possibility of overfitting by minimizing the parameters in $\hat{\boldsymbol{\beta}}$.

However, Lasso regularization, while optimizing, might set some of the parameters exactly to zero, which results in an implicit **feature selection** process and the elimination of potentially invaluable features [14].

Building up on the method introduced by Equation (1.7), it is possible to use another type of norm to penalize values of the parameters in $\hat{\boldsymbol{\beta}}$.

$$\arg\min_{\hat{\boldsymbol{\beta}}} (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\boldsymbol{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \lambda \|\hat{\boldsymbol{\beta}}\|_2^2, \tag{1.8}$$

where $\lambda \geq 0$ is again a tuning parameter and $\|\hat{\boldsymbol{\beta}}\|_2^2 = \sum_{i=1}^{p} \hat{\boldsymbol{\beta}}_i^2$ is a squared L2 norm. The approach in Equation (1.8) is then called **Ridge regularization** of linear regression. The goal of this optimization problem aligns with that in Lasso, preserving the trade-off between fit and minimal parameters in $\hat{\boldsymbol{\beta}}$.

Apart from the Lasso, Ridge generally does not make the parameters equal to zero, meaning it does not implicitly perform feature selection [14].

The most important property of these methods, along with the parameters minimization, is the model variance decrease. With a suitably chosen λ, along with the variance decrease, the bias tends to increase, which typically results in better performance on unseen data – better generalization of the model [14].

1.5 k-Nearest neighbours algorithm

The main idea, which forms the foundation of this method, is based on the assumption that feature data points residing in the given vector space, which are close to each other (for some distance definition), tend to be more similar in terms of the target variable Y [30]. This method can solve both regression and classification tasks; however, only the regression setup is explained, as only this is used in the thesis.

Suppose we have the training dataset as earlier, $\{Y_i, \mathbf{X}_i\}_{i=1}^n$, where the stacked vector $\mathbf{X}_i = (X_{i,1}, X_{i,2}, \ldots, X_{i,p})^\top$. Further, we have a new datapoint with vector of predictors $\hat{\mathbf{X}}$, but without the linked value of the target Y. Lastly, we define a distance metric $\|.\|$ and a parameter $k \leq n$.

Let $N \subseteq \{i\}_{i=1}^n$ be the set of indices of the k nearest training feature vectors $\mathbf{X}_i$ to the $\hat{\mathbf{X}}$, in terms of $\|.\|$ distance. Then, the approximation of the target Y is

$$Y \approx \frac{1}{k} \sum_{i \in N} Y_i. \tag{1.9}$$

This method is non-parametric and aims to uncover patterns solely using the given training dataset. This method is called the **k-Nearest neighbours** algorithm (k-NN).

1.6 Decision tree regression

The idea of splitting the vector space of data into subsets based on occurrence of individual data points in training dataset, as in the k-NN, is particularly easy to interpret and replicate. However, the problem might arise with uniform weighting of all predictors, as there might be predictors with larger or lower importance.

To preserve the high interpretability of space partitioning and allowing for weighting of individual predictors, data structure **binary tree** comes as a very natural candidate. The idea of using a tree is to recursively partition training dataset into tree's nodes, representing certain space partition, and effectively terminate the tree at certain depth, with leaf nodes, whose data points define the resulting target approximation.

Similarly to the k-NN case, this algorithm is capable of solving both regression and classification tasks; however, only the regression setup in explained.

Let $\{Y_i, X_{i,1}, X_{i,2}, \ldots, X_{i,p}\}_{i=1}^{n}$ be the training dataset. The complete dataset is associated with the root node of the tree structure. The tree-building algorithm aims to recursively divide this set into two disjoint subsets based on a decision criterion $X_j \leq w$ and $X_j > w$, where X_j is the certain chosen predictor among $\{X_q\}_{q=1}^{p}$ and $w \in \mathbb{R}$ specified threshold value.

The building of the tree is a greedy algorithm, which means that, at each step of the building, the algorithm chooses partition criterion locally optimally according to the current set of datapoints.

Let S be the set of datapoints in a given node. The aim is to define S_{left}, S_{right} belonging to left and right child nodes respectively, based on the division criterion. In regression, the goal is to divide S, to make S_{left}, S_{right} *as homogeneous as possible.* We can define the weighted average of target variable variances, in respective sets as

$$\text{Split}(X_j, w) = \frac{|S_{left}|}{|S|}\text{Var}(S_{left}) + \frac{|S_{right}|}{|S|}\text{Var}(S_{right}), \tag{1.10}$$

where $|S_*|$ denotes the cardinality of the set S_* and $Var(S_*)$ denotes variance of the target values within S_*. This metric is explicitly defined by S and the chosen X_j, w. With this definition, we can measure homogeneity of the resulting split, making variance of both S_{left} and S_{right} as low as possible.

This means that in all nodes of the structure, the algorithm solves

$$\arg\min_{X_j, w} \ \text{Split}(X_j, w) \tag{1.11}$$

and performs the split until a termination criterion is satisfied (e.g., maximum depth of the tree, minimum points in leaves, etc.).

With such a data structure built, a certain input $\hat{\mathbf{X}}$ belongs to particular leaf node. The approximation of Y corresponding to the $\hat{\mathbf{X}}$ is then the arithmetic mean of the leaf's training target values. This approximation method is called the **decision tree algorithm** [14].

Building an approximation solely on one decision tree might result in overfitting. Since a single tree is built using greedy approach, even a small perturbation of the training dataset might potentially result in substantially different tree structures and, consequently, very different predictions.

This sensitivity arises from the locally optimal splits in nodes, which depend heavily on the sample of data used in training. As a result, trees tend to overfit the data, particularly at relatively large tree depths [14, 30].

A possible solution comes from the statistical method of **bootstrap** – artificial samples are generated from the original data by sampling with repetition [30]. We define B as the number of such bootstrap samples, and single decision trees are trained on these samples. Then, each tree outputs a target approximation for the input as $\hat{Y}^{(b)}$. Then, the final approximation $\hat{Y}$ is

$$\hat{Y} = \frac{1}{B} \sum_{b=1}^{B} \hat{Y}^{(b)}.$$

This resampling strategy introduces diversity among individual trees, which is essential for reducing the variance in the final model [14, 30].

1.7 Time-series analysis (ARIMA)

In all of the models introduced earlier, the idea of training the model was to tune the model to be able to generalize the input variables into output, the target variable. For the purposes of predicting the future developments of time series, there exist specialized approaches beyond the mentioned conventional ML frameworks [5]. One of the most widely used frameworks is the **Autoregressive Integrated Moving Average** (ARIMA) model.

The ARIMA model relies on capturing short-term and often temporal patterns in the time series. Apart from the ML algorithms explained earlier, ARIMA is time-aware, meaning its framework directly works with the concept of time flow and the dependency of target variable on it [5].

Let there be a univariate time series of length n as $\{Y_i\}_{i=1}^{n}$. The idea of the ARIMA model relies on the concept of autoregressivity, which means that future data points of

the time series are dependent on the earlier points in series. Rigorously speaking, let there be $p \in \mathbb{N}_0$ and

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p} + \epsilon_t, \tag{1.12}$$

where ϵ_t is the irreducible noise term and $\phi_1, \phi_2, \ldots, \phi_p$ are **autoregressive coefficients**. Time series tend to be non-stationary, which means that their statistical properties tend to change over time. For this purpose, let there be an operator B, which satisfies $BY_t = Y_{t-1}$. Then, the term of **differencing** of the d-th order is defined as

$$\Delta^d Y_t = (1 - B)^d Y_t. \tag{1.13}$$

In the specific case, when $d = 1$, such differencing yields the first-order difference $\Delta^1 Y_t = Y_t - Y_{t-1}$. Differencing removes the trend of the time series, stabilizes it, and decreases its non-stationarity.

Lastly, the idea of ARIMA relies on the property of the **moving average**, which means that ARIMA models the value of Y_t as a linear combination of forecast errors made in the past:

$$Y_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \ldots + \theta_q \epsilon_{t-q}, \tag{1.14}$$

where $\theta_1, \theta_2, \ldots, \theta_q$ are **moving average coefficients**. ARIMA tends to incorporate deviations encountered in the past to improve future predictions.

By combining properties introduced in Equations (1.12), (1.13) and (1.14), and using the chosen parameters p, d, q, we obtain the complete model defined as $\text{ARIMA}_{(p,d,q)}$ [5]:

$$\Delta^d Y_t = \phi_1 \Delta^d Y_{t-1} + \ldots + \phi_p \Delta^d Y_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \ldots + \theta_q \epsilon_{t-q}. \tag{1.15}$$

The model relies on the assumption that $\epsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. Under this assumption, the joint likelihood of the observed time series can be written as

$$\mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2 | Y_1, Y_2, \ldots, Y_T), \tag{1.16}$$

where $\boldsymbol{\phi} = (\phi_1, \phi_2, \ldots, \phi_p)^\top$ and $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_q)^\top$. These parameters are estimated by solving

$$\arg\max_{\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2} \mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}, \sigma^2).$$

Such a model is then capable of predicting the future development of the time series variable, solely using its past development.

Chapter 2

Stock market data

The aim of this chapter is to introduce a set of variables related to the financial market, their possible importance in terms of predictive modeling, and lastly their subsequent processing into specific forms, which were used throughout all the related analyses in this thesis.

Market itself and particularly the price movement of financial instruments is a complex and counterintuitive process with a large number of various related variables, which contribute to price movements. In terms of using such data, as an auxiliary source for decision making in investment strategy creation, there is a rich universe of sources dealing with the issue [9, 22].

In the thesis, we aim mainly to research the predictive power of various diversely sophisticated methods of machine learning on the particular market segment. Many papers focus on a single stock or a very limited set of a few stocks [13, 35]. Others apply predictive modeling methods to stock indices (e.g., US S&P 500 and NASDAQ Composite or German DAX) [28] however, there is a small number of sources dealing with a market segment, analyzing single stocks included in it.

Our main focus is on the index S&P 500 – a US stock index accumulating the 500 biggest publicly traded companies in terms of market capitalization. The index itself is considered to be the most trustworthy general indicator of the overall US and world's stock economy state [16]. The index itself is divided into several general market segments to which stocks belong. In this thesis, we focused on the segment **Information technology**. However, the approach used in our analyses is adaptable to any subset of stocks – the only determining constraint is the respective database of stocks.

As of late 2025, the Information technology (IT) segment consisted of 68 US-traded stocks. However, we excluded three stocks (tickers APP, CRWD and DDOG), as their brief period (at most 3 years) of trading yielded insufficient historical data for reliable modeling. In the year 2025, in terms of total capitalization, the IT group makes a

total of nearly 30 % of the index, which makes it the most weighted sector, despite making only 13.6 % of all included stocks [24]. This recent growth of its weight could be explained by a lot of factors steering the market nowadays, but the general view on the topic suggests that recent boom of artificial intelligence could be the most dominant engine of growth. This suggests that focusing on this segment is reasonable, as it could be considered the most important one nowadays.

2.1 Variables in financial data

At first, we introduce specific attributes which define the value movement of all financial and economic instruments used throughout the thesis as features and target variables. These attributes could be further generalized onto any financial instrument attribute's movement, not only to price itself – as it will be applied to non-stock data as well.

For the purposes of defining desired terminology, let's define a time interval on which we analyse an instrument's attribute movement as $[t_1, t_2]$, where naturally $t_1 < t_2$. The terminology, could be universally defined for any time interval with various frequency.

At the breakpoint t_1 we define the **opening value**, which denotes the value of the attribute at the very beginning of the interval. For the purposes of this thesis, where we aim to use a daily frequency of data, this value denotes the value of the attribute at the very start of the trading day. In the opposite manner, at the breakpoint t_2 we define the **closing value**, denoting the final value of the attribute at the end of the observed interval – in our case, at the end of the trading day.

During the process of value evolving throughout the interval, we observe the global minimum and maximum of the attribute's value, which we denote **low value** and **high value**, respectively. Such data give us an indication of volatility during the interval and subsequently the sentiment of the market regarding the attribute.

The visualization of the terms for a sample interval of a Microsoft stock between the beginning of June 2025 and the beginning of September 2025 is pictured in Figure 2.1.

Last but not least, we accumulate data regarding the number of an instrument's stocks traded (either sold or bought) during the time interval. This is denoted as **volume**, and in terms of the thesis, the value of this attribute is known for stocks only; for other instruments we use the value of zero.

The last data attribute, applied only on stocks' data as well, is volume of **dividends** paid out. Certain companies oblige themselves to pay parts of their profit to their investors on a regular basis (e.g., Apple, Oracle) [6]. Some companies do not pay out dividends, even though the attribute was included in the database for all stocks.

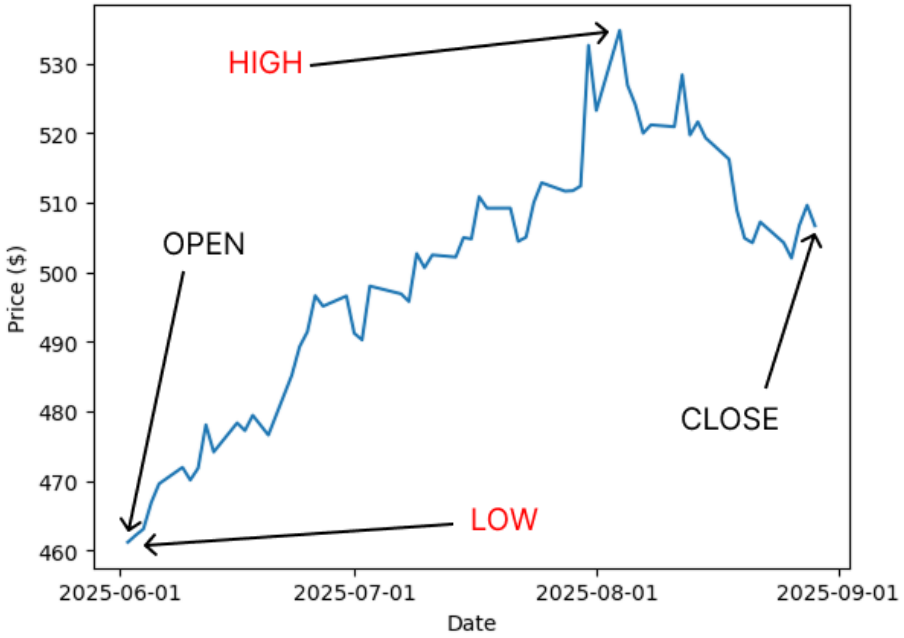

Figure 2.1: Visualization of the defined attribute's movement terms on Microsoft stock price.

2.2 Rolling window and prediction methodology

There are several approaches for splitting time-series data into training, validation, and test sets. For the purposes of next-day prediction — where data from trading days $t-q, t-(q-1), \ldots, t-1$ (for some q) are used to predict the closing price on day t — the concepts of rolling window methodology are adopted [5].

In our approach, we define a static time-series interval, which serves as a hybrid train-validation time interval. Let's define the overall data interval being between T_1 and T_2 timepoints (let $T_2 - T_1 = M$). On this interval reside two frames of variables.

Multivariate $\mathbf{X} \in \mathbb{R}^{M \times F}$, where F denotes the number of features used, which serves as an input variable. Included features in $\mathbf{X}$ are values of selected financial variables, shifted backwards up to $q = 9$ days. This means, that $\mathbf{X}[t]$ (the t-th row of $\mathbf{X}$) represents data available through days $t-1, t-2, \ldots, t-9$. [1] To ensure that the data in the $\mathbf{X}$ are not biased, each particular row is transformed via Z-score standardization.

Secondly, univariate $\mathbf{Y} \in \mathbb{R}^M$ accumulates values of the target variable (next closing price) of the particular stock, to the respective rows of $\mathbf{X}$. This means, that Y_t (the

[1] We chose the value of q due to few reasons constant. Firstly, variable q would result in larger time-complexity and necessity to optimize new parameter further. Secondly, we want to emphasize short-term market dynamics, which are commonly observed in financial time series and documented in the literature [5].

t-th value in the Y) is the value of the closing price on day t.

Further, we define parameter $1 \leq p < M$, which is the length of the training subwindow, meaning how many *next-day* predictions model would be trained on, and w, which denotes the forecasting horizon, i.e., the number of time units into the future for which predictions are made. For the purposes of this thesis and its next-day prediction methodology, we define $w = 1$.

With respect to the adopted methodology, we can generalize the process of train and validation sets split, by creating a pseudocode demonstrating the relationship between input and target matrices. Algorithm 1 shows what parts of the global $\mathbf{X}, \mathbf{Y}$ are directly available at the certain moment of the model building.

Algorithm 1 Rolling-window on (T_1, T_2) interval

```
# We assume w ← 1
# The prediction target day is t + p + 1, we have available data up to day t + p
Set look-back window length p
Retrieve the feature variables' values into the X
for each stock in database do
    Retrieve closing prices into the target Y
    for t = T_1 to T_2 − (p + 1) do
        X_train ← X[t : t + p]        # Z-score scaled
        Y_train ← (Y_t, Y_{t+1}, ..., Y_{t+p})
        X_predict ← X[t + p + 1]    # Z-score scaled
        # X_predict corresponds to the latest available data
        Train model f̂ on (X_train, Y_train): so Y_train ≈ Ŷ_train = f̂(X_train)
        Generate prediction: f̂(X_predict)
    end for
end for
```

From Algorithm 1 a few important properties of this method could be derived. Firstly, respective intervals are overlapping and the shift between respective input and target matrices, at t and $t + 1$, is a single time unit. Secondly, by allowing window overlapping, we maximize the number of possible training data points, making the training-validation process possibly more robust and general. Lastly, such setup would be implemented on train-validation interval (T_1, T_2) and on disjoint test interval likewise.

The multivariate $\mathbf{X}$ contains pre-chosen shifted variables (see Section 2.3). In this thesis, time interval between September 15, 2017 and December 31, 2022 (1333 trading days) is used as training-validation interval and time interval between January 1, 2023 and January 16, 2026 (761 trading days) is used as test interval.

At this point, it is appropriate to illustrate this data preparatory setup with a specific example in order to better understand Algorithm 1. Let $p = 4$ and suppose we want to predict the closing price on January 20, 2022. We have complete data up to January 19, 2022. With fixed $q = 9$ and given p the available training and validation data points are depicted in Figure 2.2.

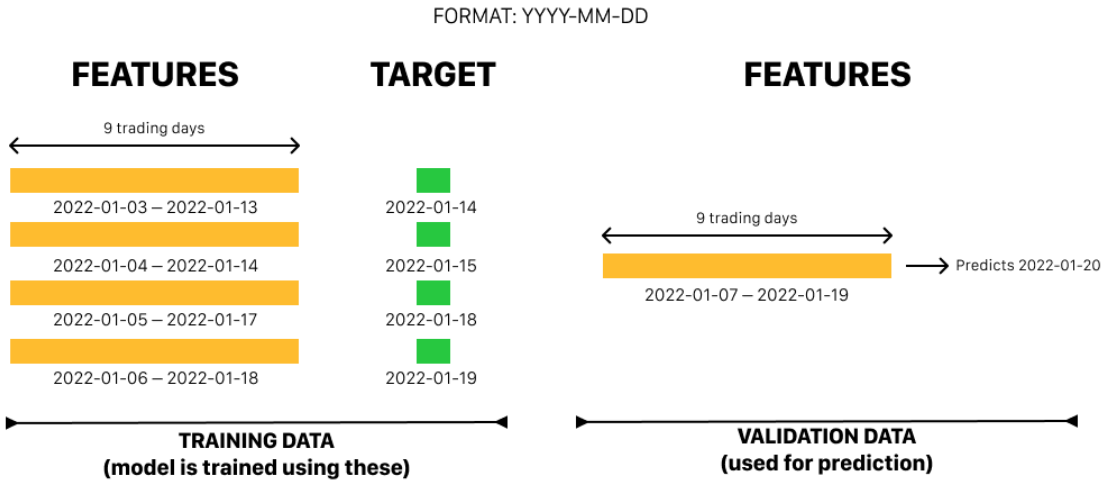

Figure 2.2: Sample algorithm step for predicting the closing price of a given stock on January 20, 2022, with $p = 4$.

Such setup is iteratively being done for every single stock, both on train-validation and test intervals.

2.3 Suitable features in modeling

The amount of variables, which are directly or indirectly generated together with day-to-day market running, is quite enormous, and one should consider subsetting them for machine learning purposes wisely. In this section, we aim to show and justify the process of our general feature selection, mostly based on *a priori* knowledge of processes within financial markets and the variable's most likely effect on stock markets.

For all of the mentioned variables in this section and for all considered shifted historical dates (up to $q = 9$ backwards), the respective **open**, **close**, **low**, and **high** values are present in the resulting feature matrix $\mathbf{X}$. On top of that, where appropriate, the **volume** and **dividends** variables are also included.

For the sake of completeness, data were obtained from the API, directly fetching data available on Yahoo Finance – available through the Python library abstraction provided by the `yfinance` library [1, 34].

2.3.1 Stock indices

In order to process the current general state of the stock exchange we researched possibilities of implementing data regarding the price development of certain stock indices. **Index** is an abstract concept of accumulating information regarding single stock assets into one unified asset, mostly using a predefined weighted average of prices of the stocks included.

The stock index is specifically designed to follow and replicate the performance of a given subset of stocks. Taking this into account, by including such well-chosen indices, which copy a broader and more various scope of stocks on a given market, we may obtain valuable information regarding the current state of the US market.

For these purposes, we decided to include two major stock indices into our database. The general index **Standard and Poor's 500**, commonly known as S&P 500, is an index including the 500 biggest US-traded stocks based on total market capitalization. The index consists of a wide scope of various segments making it robust and universal in terms of mirroring the overall US market state.

Taking into account the scope of this thesis – the Information Technology segment, we decided to include the index **NASDAQ Composite** (NASDAQ). The main difference from S&P 500 is its segment focus. NASDAQ focuses mainly on tech and IT companies, making it possibly a valuable piece of data for our database of feature data.

2.3.2 Commodities

Commodities denote mainly raw resources and materials, which are traded mainly in uniform quality and large quantities. Their fundamental role in the market comes from the fact that most of these assets play an essential part in further processing, consumption and industry. Matsumoto et al. [23] suggest, that the commodity market play a vital part in the whole financial segment, which suggests, that including such assets in the database might be sufficient.

In this thesis, we included data regarding two major commodities – **crude oil** and **gold**. Even though there might not be a clear connection to the IT sector, we believe that such data would bring valuable information about the general state of the whole global economy.

Oil plays a vital role in keeping the world in motion, and a solid majority of the world's activities depend on its presence and availability. This might imply that these dependencies on the global market would be projected into the price movement of the commodity.

Last but not least, we introduce gold price movements into the database. For a wide range of reasons, gold still remains one of the most prominent securities. Most modern currencies are dependent on a system, where gold acts as a security stabilizing

currency movements and limiting the governments issuing money. On top of that, governments rely on physical gold as a reserve in times of crises [25]. Taking these reasons into account, by including gold in the database of features, we assume we are obtaining valuable information regarding the world's economy state, crisis potential or investors' sentiment, etc.

2.3.3 Currency pairs - FOREX

Currency pairs and subsequently the **Foreign Exchange Market** (FOREX) denote a specific market, where the main traded goods are currencies. Traders involved in the market speculate on certain exchange pairs' volatility and bet on them in order to make a profit. The market itself determines price of every possible currency pair, thus making the acquisition of certain amount of currency possible through various ways – currency X could be traded through currencies Y, Z, etc., making the market far more speculative than others. By the volume of instruments traded, this decentralized market is by far the largest market in the world [19].

Being fully decentralized, meaning prices on this market are accepted worldwide, these data could be considered as a sufficient indicator of currency relative strength and, by that, an indicator of a particular economic competitiveness of a certain country. Taking this into account, we assume, that including such data inside our database would bring valuable information about the international competitiveness of US economy towards other major economies and quantitative information about relations among major economies, which is most likely an important predictor for stock price movements.

By our focus on the US market, we chose to include currency pairs with the American dollar (**USD**) in all pairs. We chose to include so-called **major currency pairs**, with respect to their overall traded volume on a daily basis, according to reports published by the Bank for International Settlements and to market conventions [2, 19].

In particular, included pairs, together with the USD, are the Euro **(EUR/USD)**, Japanese yen **(JPY/USD)**, British pound sterling **(GBP/USD)**, Swiss franc **(USD/CHF)**, Australian dollar **(AUD/USD)**, Canadian dollar **(USD/CAD)**, and New Zealand dollar **(NZD/USD)**.

2.3.4 Other market indicators

Apart from the introduced indicators and the general assets' price movement data above, there is still a vast group of possibly valuable features, which do not fit into the above-defined features' groups.

In order to track the performance of the US economy more precisely, it is wise to include information regarding national debt as possible features. For this purpose, we included data about **US Treasury bonds**. Bonds denote a financial instrument

mostly long-term, low-risk securities issued by the government of the USA, where an investor, in basic terms, lends money to the government in exchange for periodic fixed payments. These bonds are further divided based on their maturity – the time within which issuer obliges to fully repay issued money.

Price movements of bonds with various maturities are considered to be a good indicator of the US economy, and thus of the stock market state [20]. In our database of possible features we included bonds with a maturity of 13 weeks, which belong to the group of short terms bonds, and bonds with 5 year long maturity. This kind of maturity is on the edge between long-term and medium-term. The choice reflects the need to track data regarding relatively very short horizon and a long one.

Apart from the bond-related data we considered obtaining data which are not directly financial – they are not any type of asset. We researched possibilities of tracking the sentiment of investors and of the market itself. We chose to include data of **Volatility Index** (VIX), which is a specific gauge (based on S&P 500 option data) tracking state of the US stock market in terms of volatility and the sentiment of investors.

It presumably brings valuable information, which might not be included in all of the metrics described, regarding the *a priori* mood of the market and its assets' risk. Its value ranges mostly up to 80, which was considered as crisis values in years 2020 and 2008, and above approximately 10, which is considered to be relatively stable market state. Value development between the January and September of 2025 is given in Figure 2.3.

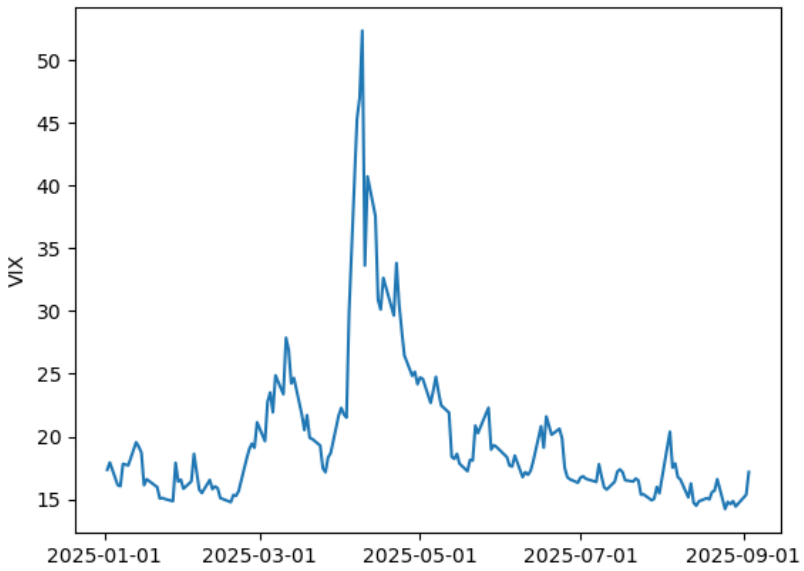

Figure 2.3: VIX value movement through the 2025.

It is surely interesting to address the sudden VIX surge during the April 2025. This sudden increase was basically day-to-day and denotes phase of repeated issuing and

cancelling of international tarrifs. As we can see, this political move damaged investors' exposure to the US market then.

2.4 Data preparation summary

Taking into account the process, how historical data are processed into the feature $\mathbf{X}$ matrix, it is possible to conclude the total number of features per single row. We consider $q = 9$ shifts backwards and in total 14 distinct financial variables. For each variable and shift, present in the database, we include in total 5 metrics (Open, Close, High, Low and Volume). This together yields in total 630 features included in the feature matrix $\mathbf{X}$.

Consequently, it is suitable to state that this feature $\mathbf{X}$ is used throughout the whole interval and for all of the analyzed stocks. The key variable in the modeling process is the target vector $\mathbf{Y}$ which consists of closing prices of the particular stock (among the 65 considered).

Chapter 3

Predictive analyses on IT segment

This chapter presents obtained modeling results. For each of the models used, in-depth parameter tuning and evaluation on both the training-validation and test datasets are performed.

In the subsequent analyzes and hypertuning, we illustrate results based on data of Enphase Energy Inc. (ENPH) stock. This stock experienced major surge and downturns in the training-validation time interval and we assume that this creates a relatively representative data sample. However, the actual tuning and conclusions were done based on all stocks, ENPH analyses illustrates the general behaviour of the models.

3.1 Evaluation metrics

Throughout the thesis, particular performance metrics are used. Suppose that on the day t we acquired closing price data denoted as Y_t; therefore, based on the methodology introduced earlier (see Section 2.2), we obtain a prediction for the day $t+1$ (using the model trained on the data available through days $t, t-1, \ldots, t-(p+1)$) denoted as $\hat{Y}_{t+1}$.

Firstly, in order to measure deviation from the actual Y_{t+1}, metric **absolute percentage error** (APE) is used. The percentage relativization was chosen in order to unify the scale among all the stocks, making the prediction error comparable across the stocks and more interpretable. The complete definition of the metric is given as:

$$\mathrm{APE}(\hat{Y}_{t+1}, Y_{t+1}) = \frac{|Y_{t+1} - \hat{Y}_{t+1}|}{Y_{t+1}}.$$

Building upon the APE definition, the average value of the APE for individual stock over the entire time interval considered is evaluated. This is further referred to as **mean absolute percentage error** (MAPE).

Apart from that, we introduce a baseline benchmark to measure the model's practical applicability. According to Fama [7], the asset price developments correspond to the random walk. In accordance with this, such naïve method can be introduced, that benchmark predicts the next-day price on day $t+1$ to be equal to the price on day t.

With the benchmark method defined, we quantify performance as the proportion of predictions made by the model that are closer, in terms of APE, to the actual closing price than the predictions made by the naïve benchmark. From now on, such benchmarking will be referred as the **model's dominance**.

3.2 Naïve model

Employed naïve benchmark model does not employ any ML framework. Benchmark assumes no return in the following day; rigorously speaking, prediction of the closing price $\hat{Y}_{t+1}$ on day $t+1$ is $\hat{Y}_{t+1} = Y_t$.

Evaluation result of the model, on the particular training-validation interval, is average MAPE (over all the analyzed stocks) of $\mathbf{1.69}\%$. In this case it has no meaning to analyze dominance, as it has no interpretative value.

3.3 Linear regression

Although linear regression might seem rather simple, compared to other conventional ML frameworks, it has been implemented in financial predictive modelling quite regularly. According to Zhao [36], linear regression is capable of reflecting the trend-based development of a certain stock up to a certain extent. Zhao suggests that linear regression alone might not reflect patterns in general for any stock.

On the other hand, Wang et al. [33] suggest that linear regression, with appropriate data preprocessing, is capable of performing short-term predictions up to a certain accuracy threshold.

Our next-day approach can be considered short-term; therefore, we decided to include this framework in the thesis. Apart from the conventional regression, we considered a possible feature selection optimization in each rolling window step and optimized the parameter p.

3.3.1 Possible feature selection

In order to analyze the possible contribution of omitting certain predictors, we implemented rolling forecasting with the selection of given k most relevant predictors with respect to the next-day closing price. Taking into account the potentially extensive

computational time of such prediction, predictor selection based on F-statistics was used.

The statistics reflects the ratio of variance explained by a single predictor (with respect to the closing price) to the remaining unexplained variance. Predictors that achieve higher values of the statistics are considered strongly correlated with the target [14]. After such precomputation, the k best-performing predictors are kept in feature matrix.

However, such an approach tends not to perform significantly better than keeping all the predictors. Figure 3.1 shows a sample example of MAPE dependence on the number of selected k best features for the stock Enphase Energy Inc. (ENPH).

It is possible to observe that including more predictors causes a decrease in MAPE. Up to a certain number k, the MAPE level tends to decrease. A further decrease in k results in a stable MAPE level (red dotted line), which represents the level of MAPE when no predictors are omitted.

Such behavior was observed in the vast majority of stocks analyzed, with only slight absolute deviations (roughly $10^{-3} - 10^{-1}\%$) below the benchmark line observed. From now on, no feature selection in terms of linear regression is used.

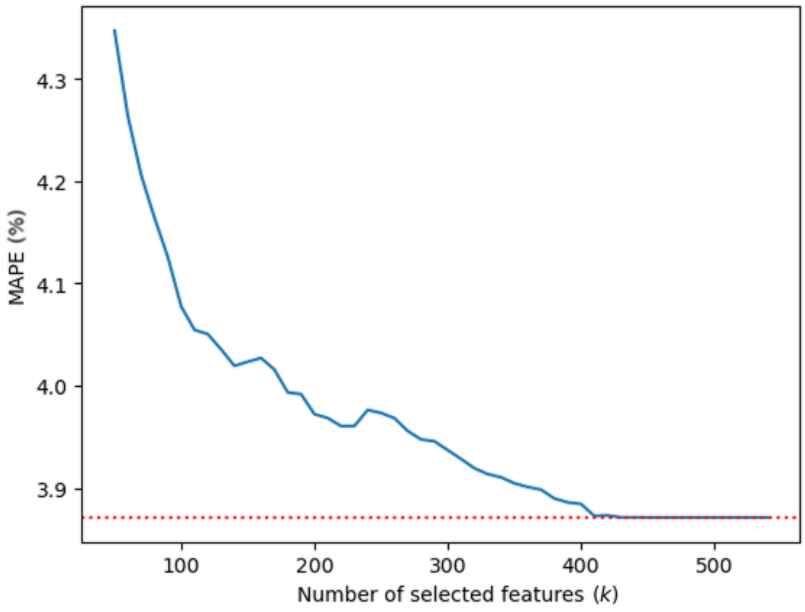

Figure 3.1: MAPE dependence on number of features selected (Linear regression, stock ENPH).

3.3.2 Optimization of p

The only *a priori* model parameter present in the linear regression is the number of observations included in the training dataset (denoted as p). Analysis of the dependence of MAPE on the parameter p for the selected ENPH stock, is shown in Figure 3.2.

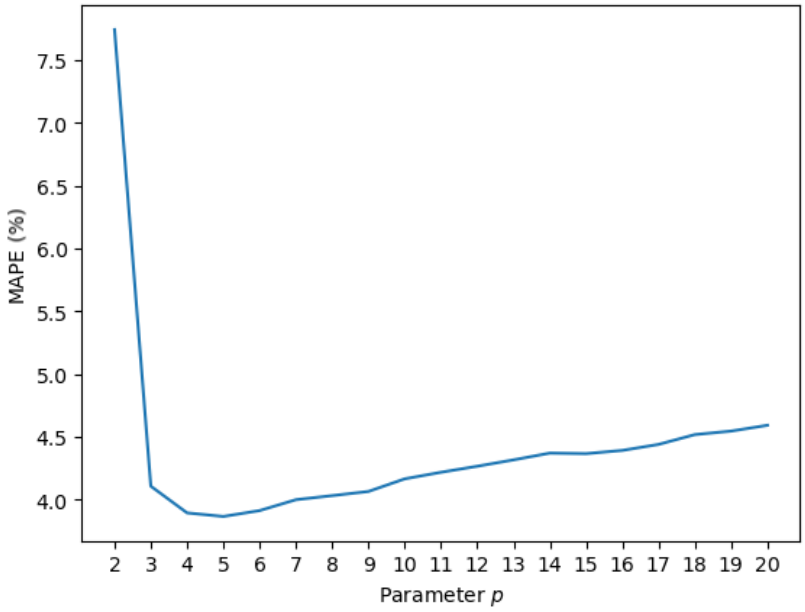

Figure 3.2: MAPE dependence on the rolling window training length parameter (stock ENPH).

In Figure 3.2, local minimum is observed at $p = 4$. In most of the included stocks we observed a similar convex development of MAPE dependence on p. What is more, the minimum, in the vast majority of stocks, is observed in $p = 4$, rarely at $p = 5$. The value of $p = 4$ was chosen, based on the overall optimization on the training-validation interval.

Such a defined model achieved (on the training-validation interval) average MAPE of **1.825 %** and dominance of **45.05 %** on average for all the stocks. In terms of interpretation, this means that linear regression is inferior, in terms of prediction accuracy, to the naïve benchmark on the given interval.

It is possible to analyze performance of the model for the given set of stocks. In Figure 3.3 is the overview of distributions of APE of predictions across all the stocks on the training-validation interval.

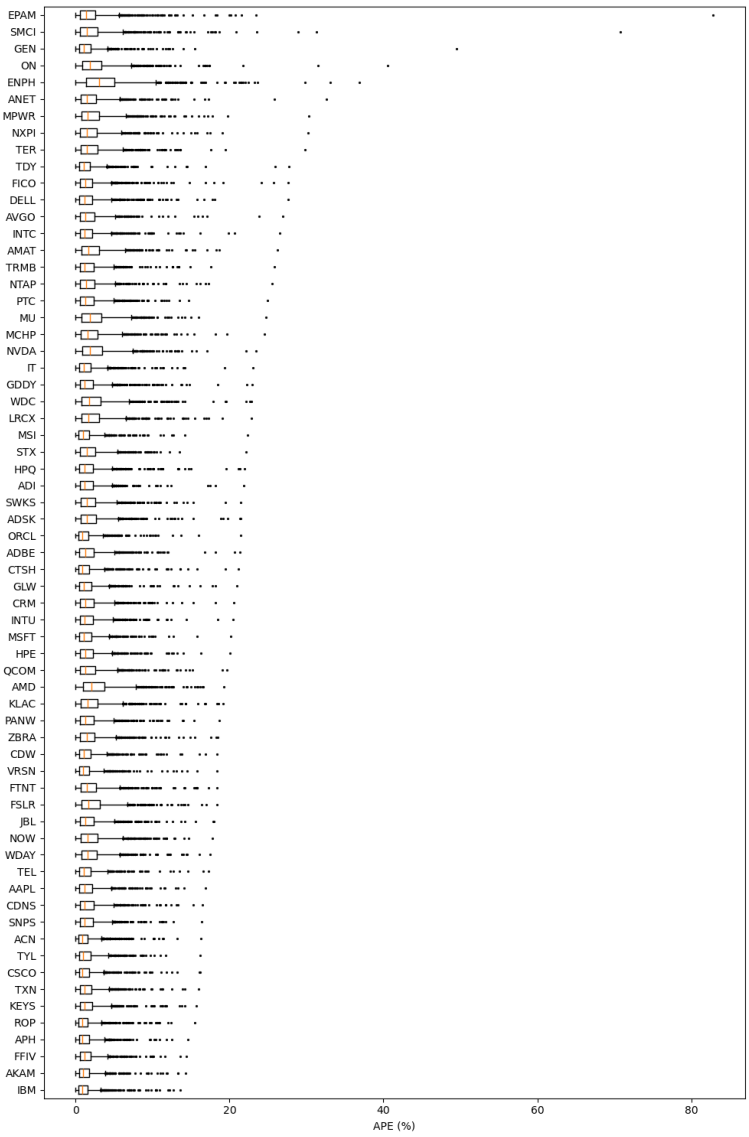

Figure 3.3: Distribution of APE for single stocks on training-validation interval (Linear regression).

In the vast majority of stocks, maximum APE of approximately 20% was observed; however, even pathological predictions with abnormally high APE were present. Even though, these cases were observed in minimal number of cases – maximal MAPE across the stocks did not exceed 4%.

3.4 Lasso regularization

Introducing regularization into the regression process might presumably improve the overall performance in terms of the introduced metrics, even though explicit feature selection has not proved to be sufficient. Apart from the linear regression framework, it is necessary to further tune the regularization parameter λ, together with the length

of the training set p.

Figure 3.4 visualizes the performance of MAPE depending on the chosen parameters λ and p for the ENPH stock.

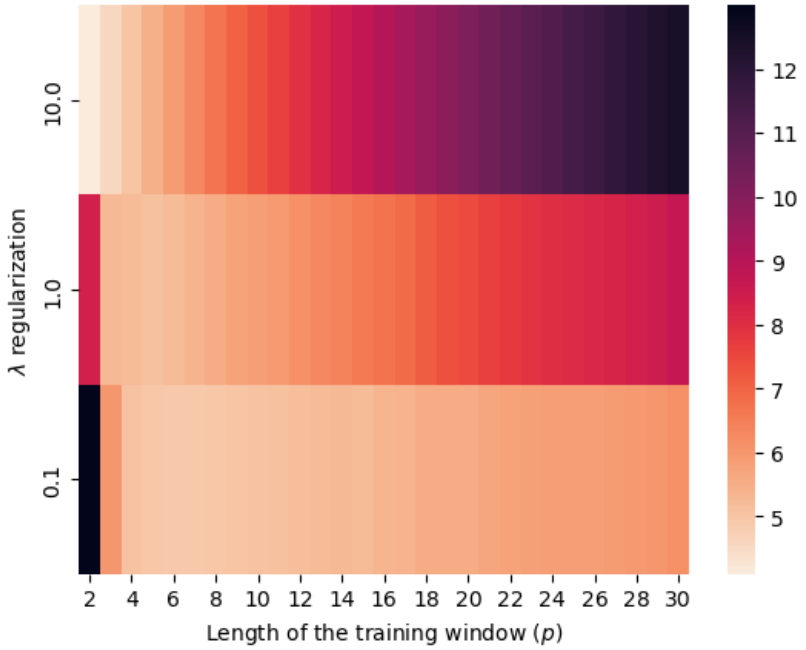

Figure 3.4: MAPE dependence on the rolling window training length parameter and regularization parameter (stock ENPH).

In terms of the parameter p, it is possible to observe that the lowest MAPE is achieved in the approximate interval $p \in [4, 8]$ for the chosen values of λ. Based on further evaluation of mean MAPE performance across all stocks, the lowest mean MAPE was achieved for $p = 4$. Further analyses were conducted with this value of p.

Subsequently, the interval of the optimal setting of λ for the given stocks was derived as $\lambda \in [4, 5]$. Figure 3.5 depicts an overview of MAPE for the chosen ENPH stock in dependence from λ on the focused interval. What needs to be pointed out is the MAPE fluctuation on the given subinterval $[4.5, 4.7]$. The improvement remains negligible, reaching at most an absolute magnitude of $10^{-3} - 10^{-2}\%$.

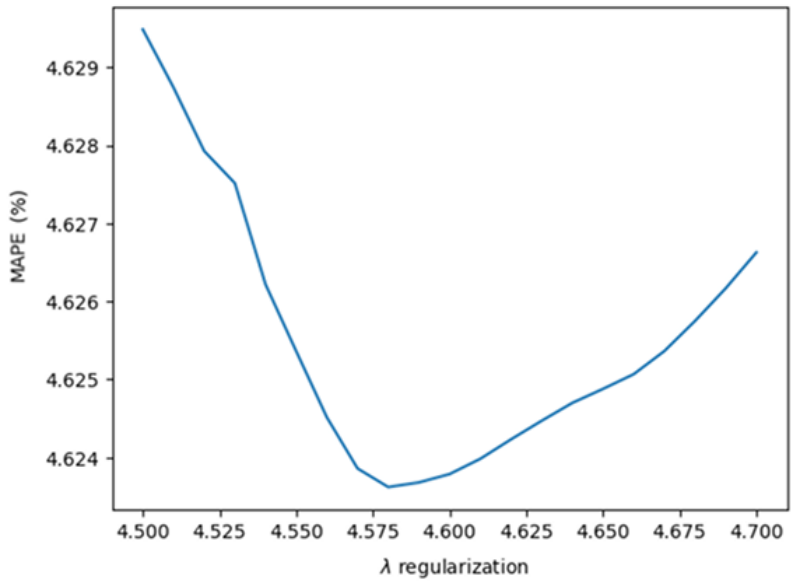

Figure 3.5: MAPE dependence on the rolling window training length parameter and regularization parameter (Lasso, stock ENPH).

With the optimized parameters, we decided to verify two approaches to setting up the model. First, we used average of optimized values of λ of each of the stocks. The resulting value is $\overline{\lambda} = 4.15$. We set up the Lasso model with the given $\overline{\lambda}$ (General model) and, in contrast, we verified performance of a model that identifies the predicted stock and applies its particular optimized λ (Stock-wise model). The results of performance on the given training-validation interval are summarized in Table 3.1.

Model Type	**MAPE (%)**	**Dominance (%)**
General model	2.297	36.162
Stock-wise model	2.291	36.156

Table 3.1: MAPE and dominance values for different settings of the model (Lasso).

Generally speaking, compared to the linear regression, there is a significant worsening in terms of both metrics. What is interesting is the behavior of both metrics. The General model has achieved comparatively worse MAPE; however, its dominance is slightly higher than that of the Stock-wise model.

Lastly, Figure 3.6 visualizes the performance of the optimized model for individual stocks. The visualization shows performance of the General model as there is no significant difference in the corresponding visualization of the Stock-wise model.

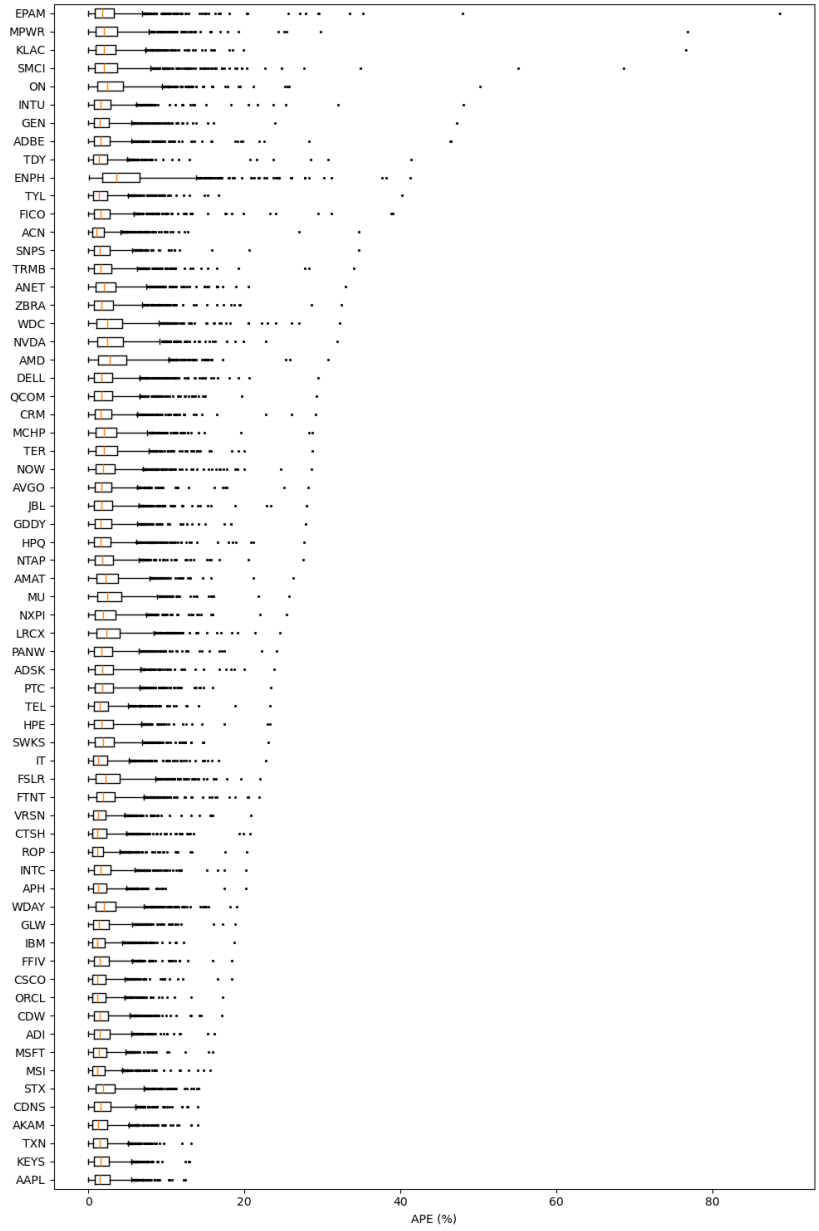

Figure 3.6: Distribution of APE for single stocks on training-validation interval (General model Lasso).

The distribution of maximal APE in Figure 3.6 seems to be very similar to the one in Figure 3.3, which visualizes distribution of APE in linear regression. Even so, there are a few minor differences – the order of the stocks, in terms of the maximal APE, is not preserved. For example, the lowest maximal APE in Lasso was achieved on AAPL stock (Apple Inc.); however, in linear regression, it was achieved for the IBM stock. This suggests that different ML algorithms most likely perform differently when predicting next-day price of single stocks – the stock having relatively the best prediction results under one algorithm does not necessarily perform best under another.

3.5 Ridge regularization

Similarly to Lasso, Ridge regularization might improve the overall performance of the regression, compared to the linear regression. The parameter tuning process in this case is analogous to the one used in Lasso (see Section 3.4).

In Figure 3.7, the dependence of MAPE for the chosen ENPH stock on the selected regularization parameter λ and the training window length p is depicted.

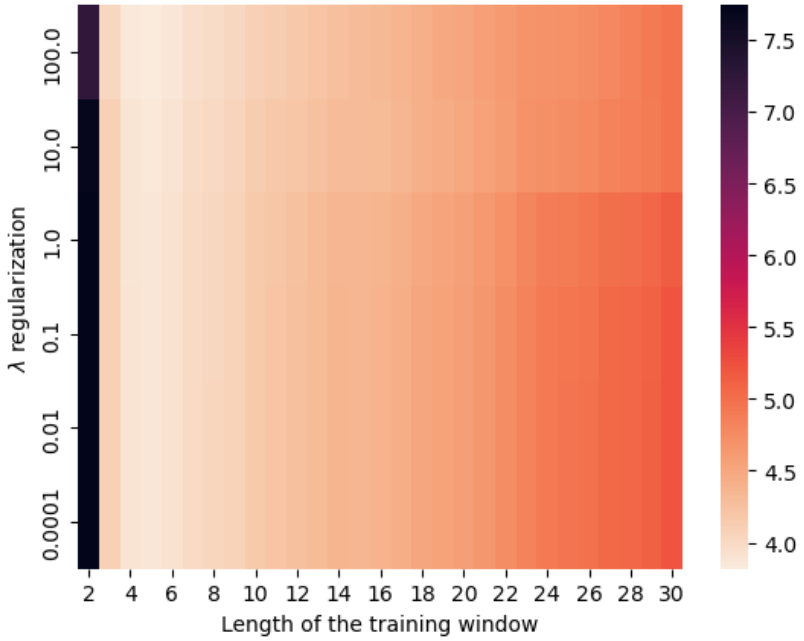

Figure 3.7: MAPE dependence on the rolling window training length parameter and regularization parameter (Ridge, stock ENPH).

Directly compared to the MAPE development in Figure 3.4, there is clear evidence of the optimal value of p being in the segment of $p \in [4, 6]$. The rise of error with the increasing value of p is much more visible and stable among various values of λ. With further experiments based on all included stocks, value of $p = 4$ was chosen for later analyses – the lowest value was chosen to save computational time. Analogous to the problem of choosing value of p in Lasso regression, a certain subinterval of near-optimal values was localized.

As seen in Figure 3.8, dependence of MAPE on the value of the parameter λ tends to follow a convex course. The segment of optimal λ for the set of stocks lies in the interval $\lambda \in [170, 260]$.

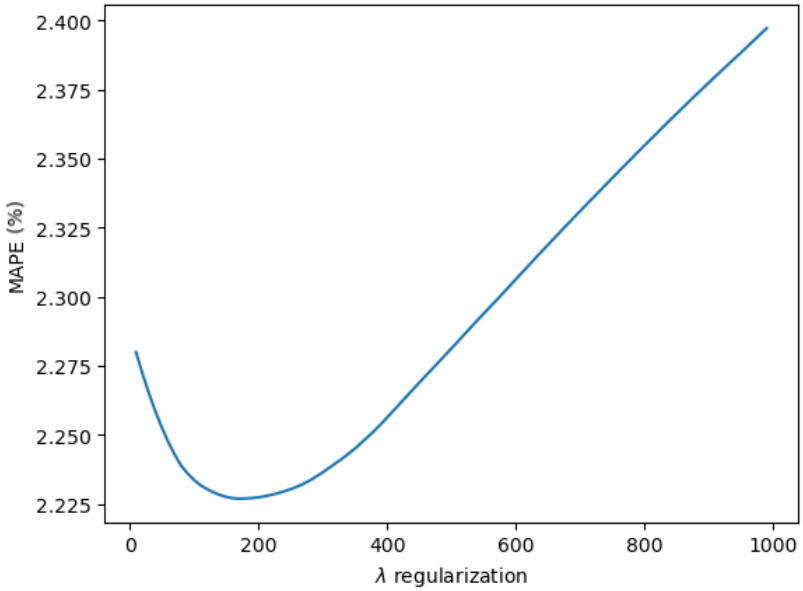

Figure 3.8: MAPE dependence on the regularization parameter (Ridge, stock ENPH).

Similar to the approach in Lasso regularization, using two model settings was chosen. The average value of optimal λ for the analysed stocks is $\overline{\lambda} = 211.85$ (General model). Similarly, a model using the optimal value of λ for each stock independently was also implemented (Stock-wise model). The final evaluation of these models is overviewed in Table 3.2.

Model Type	**MAPE (%)**	**Dominance (%)**
General model	1.784	45.3
Stock-wise model	1.783	45.43

Table 3.2: MAPE and dominance values for different settings of the model (Ridge).

Compared to the evaluation of linear regression, progress was achieved in both metrics. Ridge models have clearly better dominance and MAPE. Interestingly, compared to the case of Lasso, where counter-intuitive behavior of MAPE and dominance was observed, the opposite behavior is seen in Ridge models. General model tends to have higher MAPE and lower dominance, which is more intuitive.

Last but not least, an overview of APE distributions for individual stocks in the models are depicted in Figure 3.9 (General model). Similar to the Lasso case, the distinctions between General and Stock-wise models are negligible.

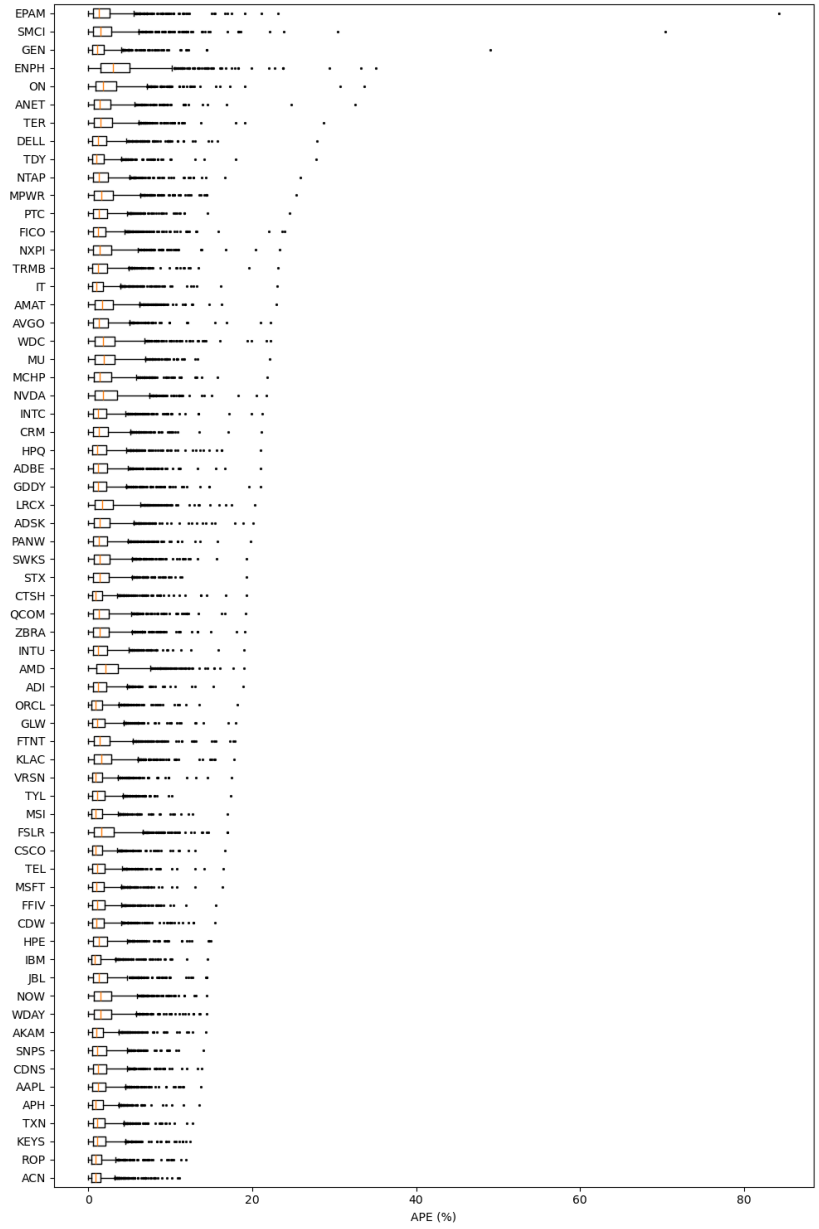

Figure 3.9: Distribution of APE for single stocks on training-validation interval (General model Ridge).

Overall, there are only minor distinctions to the presented Figures 3.3 and 3.6, mainly in the order of stocks based on the error range. This again suggests that different ML frameworks tend to have varying performance across different stocks.

3.6 k-Nearest neighbours regression

Compared to the methods used earlier in this chapter, k-NN is a non-parametric method. The effect of this property on the predictive performance is worth exploring. Although the framework seems simple and one might expect limited practical ap-

plicability, several studies demonstrate its potential and relevance in predicting stock prices.

Huang [13] included the k-NN algorithm in the stack of algorithms applied on stock price prediction. Even though k-NN did not prove to be the best-performing among the algorithms used, Huang states that k-NN performs better in terms of the employed evaluation metric than more complex support vector regression framework.

Further, Yin et al. [35] showed that the k-NN algorithm tends to perform well, as their analysis of certain US stocks demonstrated that the deviation of predictions is relatively low, compared to other frameworks even for longer forecast periods.

In the case of this algorithm, it is necessary to analyze the combined effect of choosing both parameters k (number of neighbours) and the length of the training window p. Figure 3.10 shows the effect of dependence of MAPE on given values of parameters. The visualization is based on the ENPH stock.

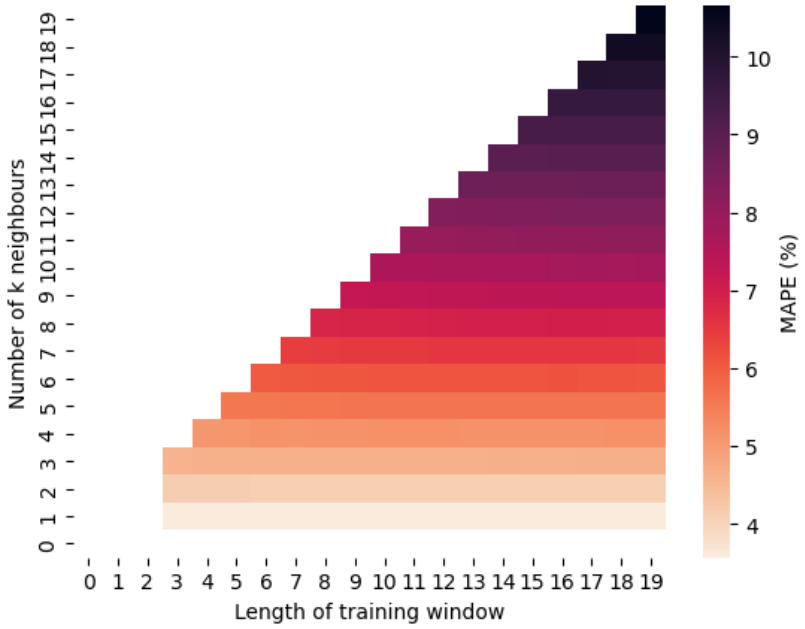

Figure 3.10: Dependence of prediction MAPE on k and p parameters (k-NN).

There is a clear trend showing that the most influential parameters is k, number of neighbours. The most interesting observation is the almost constant MAPE across all the considered values of p (with constant k). This appears to be highly counter-intuitive quality; however, further analyses have shown that it could be explained as a form of pathological behavior of k-NN algorithm.

Across all stocks, k-NN tends to reach its optimal setting when $k = 1$. In all rolling window steps, across all the stocks, the closest 'neighbour' to the prediction input is the observation from the **most recent previous day**. As k-NN has optimal setting in $k = 1$ regardless of p, k-NN algorithm effectively performs as the proposed

naïve benchmark model. Consequently, k-NN cannot outperform it. Such setting ($k = 1, p = 4$) has achieved average MAPE across the stocks **1.69 %** (the same as in the naïve model).

To complete the analysis, Figure 3.11 depicts the distribution of APE across the stocks.

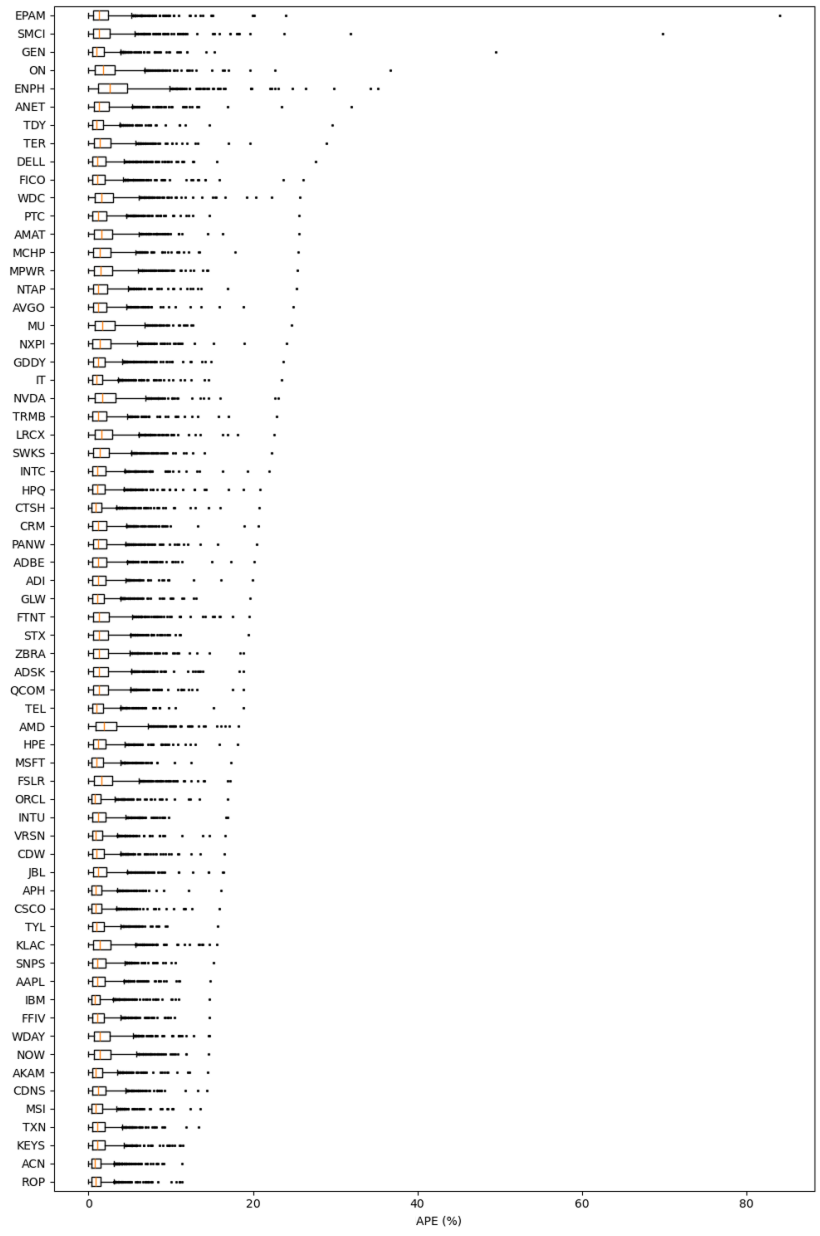

Figure 3.11: Distribution of APE for single stocks on training-validation interval (k-NN).

Again, the similar property, regarding the distributions of APE, is observed here as well. The individual APE distributions tend to be very similar to ones analyzed earlier; however, the order of stocks based on maximal APE is slightly shuffled. Interestingly, the order of stocks is very similar to the one in Figure 3.9 (Ridge). This may root

from the fact that these models are very close to each other in terms of evaluation performance.

3.7 Random forest regression

The Random forest regression framework is a frequently adopted algorithm in the finance area. There are studies employing it to predict the next-day price developments. Khaidem et al. [17] implemented Random Forest in its classifier form to predict direction of change of the next-day price. Using specific feature engineering, the authors suggest that this framework is capable of outperforming random guessing.

However, there are few papers adopting the regression form of Random Forest for stock price prediction. Krauss et al. [18] proposed an ensemble implementation of Random Forest in combination with other algorithms, but reported performance only on the S&P 500 index.

In the analyses conducted in this thesis, Random Forest Algorithm (RFA) aligns with many phenomena observed earlier. In Figure 3.12, the dependence of MAPE, for a single stock, on the parameter p – the length of the training window – is shown.

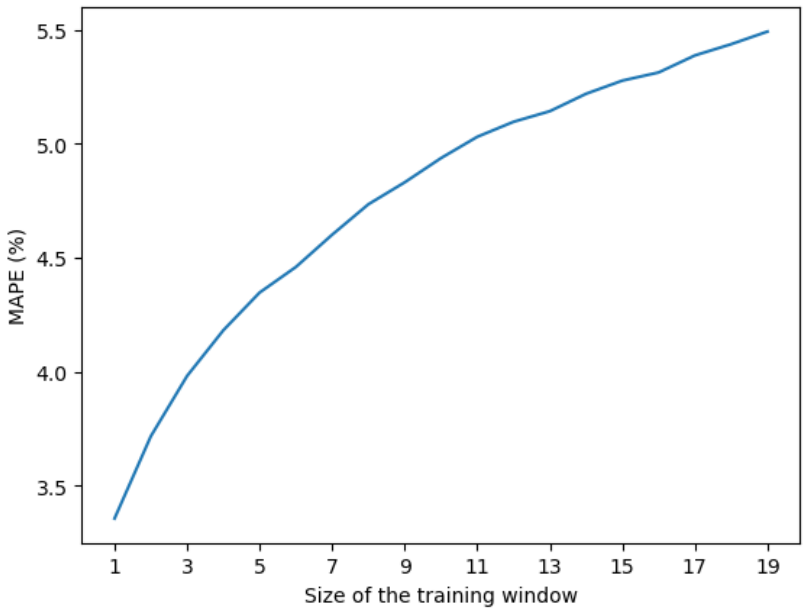

Figure 3.12: Dependence of MAPE on the length of the training window p (Random Forest, stock ENPH).

A distinct feature, similar to the one explored in Section 3.6, can be observed. For all individual stocks in the datasets, RFA tends to perform the best, in terms of MAPE, when $p = 1$. Effectively, this means that training phase consists of only the most recent known data entry – the previous trading day. Consequently, RFA tends to fall into explained pathological case, equivalent to the proposed naïve benchmark.

Moreover, tuning the available parameters (e.g., number of decision trees in the forest, maximum depth of single trees, etc.) does not change the course of parameter optimization. In all analyses and across all analyzed stocks, the model setting $p = 1$ performs the best. In addition, for settings $p > 1$, there is no improvement in dominance compared to the approaches presented above using other algorithms. Figure 3.12 shows dependence of model dominance on the parameter $p > 1$ – the best setting in terms of the dominance is $p = 2$.

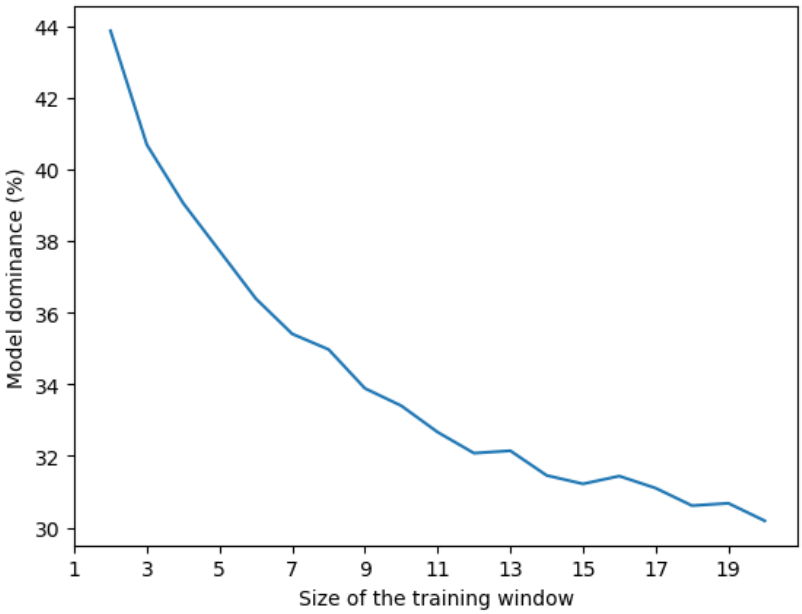

Figure 3.13: Dependence of model dominance on training window length (stock ENPH).

According to the analyses, this means that evaluation of MAPE is the same as in the k-NN framework and naïve model, as the setting $p = 1$ was chosen. The value of average MAPE per stock (in the optimal model) is **1.69 %**. The respective distributions of particular APEs is equivalent to the Figure 3.11.

3.8 ARIMA model

As mentioned in Section 1.7, ARIMA framework is capable of capturing time dependencies by default. This can be beneficial for predictive analysis in this context. Apart from the previous ML models, in this case ARIMA is trained only using univariate time series of closing prices. Due to the potentially high time complexity, features were omitted from training. In order to tune parameters p, d, q of the ARIMA model, an approach based on the principles of grid search was used.

For each particular rolling window, initial state is set as $p = 0, d = 0, q = 0$.

Incrementally, algorithm evaluates neighboring settings, such as $p+1, q+1, d+1$. For each of the combinations (up to the defined upper limit of each parameter), model is fitted according to the maximum likelihood estimation (according to the Equation (1.16)). For each fitting, Bayesian information criterion (BIC) is evaluated and finally, based on this metric, the best-performing setting is selected [31]. This is a similar hypertuning method to the ones applied before.

This means, there is no need to present any dependence of MAPE or dominance on the chosen model parameters. For each rolling window and for each stock within it, the optimal settings of p, d, q are chosen based on BIC and there is no need to optimize parameters separately or visualize dependencies similar to those in previous sections.

Lastly, it is suitable to outline the use of the parameter p (length of the training window). Its value was set constant over the entire interval as $p = 20$ and the ARIMA tuner was bounded not to analyze past values beyond this timepoint. What is more, it is suitable to state a denotion conflict, as p is chosen to denote training window length and ARIMA autoregressivity parameter. According to the traditional marking used in context of ARIMA, we decided to maintain it.

On the training-validation interval, this optimized model achieved an average MAPE **1.897 %** and dominance of **58.25 %**. Although the model was not able to outperform naïve benchmark or even Ridge in terms of average MAPE, it performed best overall in terms of model dominance. Such result is particularly interesting, as a comparatively lower MAPE does not necessarily imply higher dominance.

On the other hand, this dominance result might arise from the fact that in approximately 42.8% of rolling window steps across the stocks, p, d, q optimization terminated in a pathological case where $(p, d, q) = (0, 1, 0)$, which is equivalent to the proposed naïve benchmark method.

To complete the picture of performance, distribution of APE across the stocks is depicted in Figure 3.14.

At this point, it is possible to clearly outline that performance of given ML frameworks varies from stock to stock. As stated before, stocks tend to have various predictability in different ML algorithms, even though input features remain the same. On the other hand, it is possible to observe one common feature among all APE distribution visualizations. In all analyzed distributions, three common stocks are present in the top ranks in order of maximal APE – particularly EPAM Systems Inc. (EPAM), Super Micro Computer (SMCI) and Gen Digital Inc. (GEN).

In all these stocks, a few model predictions were significantly off, with APE exceeding 40% (at most over 80%). Such large deviations typically occur during periods of abnormal price movement or heightened volatility. In these situations, the underlying dynamics of the stock price change abruptly and cannot be adequately captured by

models trained primarily on historical patterns. However, such cases represent small fraction of events on the market and should be interpreted as outliers and not systematic bias.

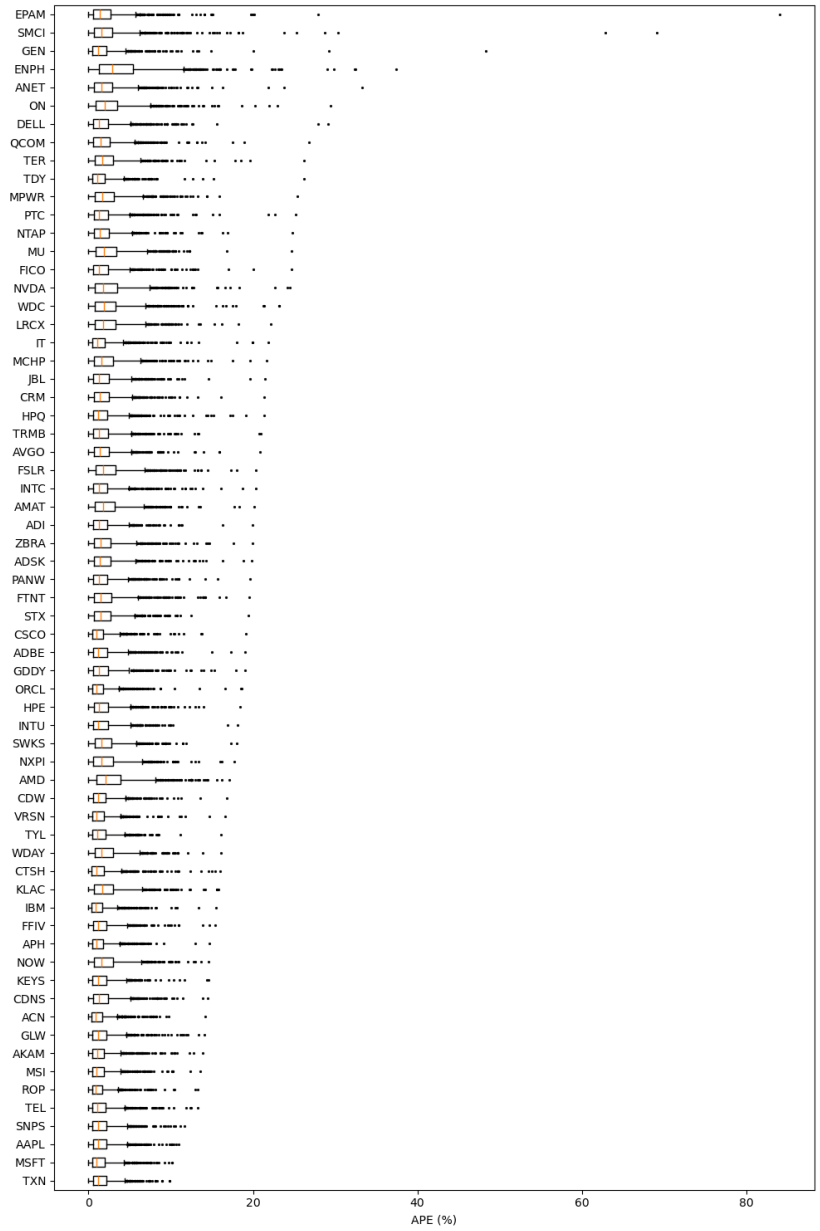

Figure 3.14: Distribution of APE for single stocks on training-validation interval (ARIMA model).

3.9 Test evaluation

In the previous sections, the process of parameter tuning was outlined in detail and, based on the introduced metrics, evaluated in order to optimize model performance. As introduced earlier, the same rolling-window-based prediction approach is used on the test interval as well (see Section 2.2).

The evaluation of all implemented algorithms on both training-validation and test interval is shown in Table 3.3. Values of model dominance for k-NN and RFA are blank as they have no interpretative value – models are equivalent to the naïve benchmark.

Model Type	**MAPE (%)**		**Model Dominance (%)**	
	Training-validation	Testing	Training-validation	Testing
ARIMA	1.897	1.923	58.25	56.68
Ridge (Stock-wise model)	1.783	1.918	45.4	42.57
Ridge (General model)	1.784	1.904	45.3	43.48
Linear regression	1.825	1.903	45.05	43.89
Lasso (General model)	2.297	2.18	36.16	39.12
Lasso (Stock-wise model)	2.291	2.36	36.16	36.55
Benchmark (Naïve, k-NN, RFA)	1.69	1.72	–	–

Table 3.3: Evaluation of tuned models on the training-validation and the testing intervals.

As seen in Table 3.3, in both cases the metrics tend to reach similar values. This suggests that the models have not suffered from overfitting. Apart from that, it is interesting to observe similar values of both metrics on differently wide intervals. This may indicate that models are relatively stable for various forecast horizons.

Chapter 4

Next-day optimization of portfolio

The goal of this chapter is to implement obtained prediction models and utilize their prediction ability to construct an investment portfolio management framework. Within this chapter, an investment portfolio refers to the allocating capital among particular stock assets in order to achieve various objectives (e.g., maximize return, minimize volatility, etc.).

4.1 Markowitz mean-variance portfolio

Allocating capital to a given set of stocks in order to balance the trade-off of asset volatility and portfolio return can be formulated as an optimization problem. There are several approaches to properly constructing a portfolio strategy, each requiring different types of information from the data [26].

For the purposes of this thesis, together with the obtained next-day price prediction of individual stocks, as the most suitable framework can be considered **mean-variance portfolio** (MVP), introduced for the first time in 1952 by Harry Markowitz [26].

Suppose $\mathbf{w} \in \mathbb{R}^a$, where a is the number of assets considered, is a vector of weights, and $\mathbf{w}_i$ represents fraction of capital assigned to the i-th asset. Further, let $\boldsymbol{\Sigma} \in \mathbb{R}^{a \times a}$ denote the covariance matrix of asset returns in the given time interval. Lastly, let $\boldsymbol{\mu} \in \mathbb{R}^a$ represents the vector of expected returns of considered assets in the next time unit.

The goal is to balance the overall return of the portfolio (represented as $\mathbf{w}^\top \boldsymbol{\mu}$) with the portfolio variance (represented as $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$). Such a bi-objective optimization problem can be formulated as follows:

$$\max_{\mathbf{w}} \quad \mathbf{w}^\top \boldsymbol{\mu} - \lambda \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}, \tag{4.1}$$

where λ is a hyperparameter controlling the investor's aversion to the risk. This problem aligns with the assumption that higher returns yield higher risk [26].

For the purposes of this thesis, it is sufficient to introduce certain constraints in (4.1). In the considered strategy, short selling of stocks (seeking profit from an asset's decline) is forbidden (represented as $\mathbf{w} \geq \mathbf{0}_a$). Lastly, we require that the entire capital is allocated, with no uninvested funds (represented as $\mathbf{1}_a^\top \mathbf{w} = 1$).

However, framework in (4.1), together with the introduced constraints, tends to be impractical in real-world scenarios [26]. The problem arises with frequent re-allocations and transaction costs in practice. Particularly, frequent re-allocations generate higher costs for the investor and subsequently lead to a decline in profit. With next-day predictions considered in this thesis, we decided to maintain the daily re-allocations; however, to minimize day-to-day transactions, we adopted an *ad hoc* modification of the framework based on Equation (4.1).

Consider the variable $\mathbf{w}_t \in \mathbb{R}^a$ representing the portfolio allocation weights on day t and $\boldsymbol{\mu}_t \in \mathbb{R}^a$ being the expected returns of the assets on day t. Taking into account Equation (4.1) and the objective of minimizing transactions, it is possible to define the optimization problem of capital allocation on day t as follows:

$$\begin{aligned} \max_{\mathbf{w}_t} \quad & \mathbf{w}_t^\top \boldsymbol{\mu}_t - \lambda_{\mathrm{R}} \mathbf{w}_t^\top \boldsymbol{\Sigma} \mathbf{w}_t - \lambda_T \|\mathbf{w}_t - \mathbf{w}_{t-1}\|_1 \\ \text{Subject to:} \quad & \mathbf{w}_t \geq \mathbf{0}_a \ \text{ and } \ \mathbf{1}_a^\top \mathbf{w}_t = 1, \end{aligned} \tag{4.2}$$

where λ_R denotes risk-aversion hyperparameter and λ_T stands for approximate cost of a single transaction [26]. This modified MVP framework (with constant $\lambda_T = 0.01$) will be used in all analyses presented in the chapter.

4.2 Portfolio evaluation metrics

In order to evaluate success and to compare different portfolio-building settings, it is necessary to define comparative metrics that measure specific qualities of a portfolio.

Suppose strategy is applied over a time interval of length N. We can track the value of the portfolio over this interval; let the portfolio values be represented as a discrete time series $V = V_1, V_2, \ldots, V_N$.

To measure the average percentage return per year, we can define:

$$\mathrm{CAGR}(V) = \left(\frac{V_N}{V_1}\right)^{\frac{1}{r}} - 1, \tag{4.3}$$

where r is the number of trading years between V_1 and V_N. The metric in Equation (4.3) represents the **cumulative annual growth return** (CAGR) [26].

In order to quantify volatility of the strategy, let $r_t = \frac{V_t}{V_{t-1}} - 1$ denote the return on day t. Volatility over a given period is given as a sample standard deviation of

$r_1, r_2, \ldots, r_N$ defined as $\sigma = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(r_t - \overline{r})^2}$, where $\overline{r}$ denotes the sample mean of $r_1, r_2, \ldots, r_N$. To annualize this metric, we define:

$$\mathrm{AV}(V) = \sigma\sqrt{P}, \tag{4.4}$$

where P is the number of periods (trading days) per year. This metric is referred to as **annualized volatility** (AV) [26].

Last but not least, it is possible to define a compound metric that takes into account both Equations (4.3) and (4.4). Let us define:

$$\text{Sharpe ratio}(V) = \frac{\mathrm{CAGR}(V)}{\mathrm{AV}(V)}, \tag{4.5}$$

which is referred to as the **Sharpe ratio** and represents the return the strategy earns per unit of risk [26].

4.3 Evaluation on the training-validation interval

Although a total of eight model variants were considered, only a subset of them was included in the portfolio-building analyses. Naturally, we chose ARIMA model, as its dominance was the highest among all the models. Further, we decided to include both Ridge and Lasso (in their General model form). These two were selected to analyze the performance of two significantly different model frameworks (in terms of both MAPE and model dominance).

In all subsequent analyses, the main focus is on tuning the risk-aversion hyperparameter λ_R (with constant $\lambda_T = 0.01$). The same data split as in Chapter 3 is applied – training-validation (between September 15, 2017, to December 31, 2022) and testing (between January 1, 2023, to January 16, 2026) intervals.

In **model-backed portfolio** optimizations, the expected return vector $\boldsymbol{\mu}_t$ for a given day t is derived from the tuned rolling-window prediction models introduced in the previous sections (see Chapter 3).

In addition, a specific approach is required for deriving the covariance matrix $\boldsymbol{\Sigma}$. Suppose we are predicting the closing price on day t, with historical data for days $t-1, t-2, \ldots$ available. Subsequently, percentual returns for each stock, on each of the days $t-60, t-59, \ldots, t-1$ are calculated. Using these vectors of historical returns for each stock, the covariance matrix of stock returns is computed, which serves as $\boldsymbol{\Sigma}$.

A constant 60-days backward window was chosen as a compromise between having a sufficiently large sample for reliable covariance estimation and maintaining responsiveness to recent market conditions.

4.3.1 Naïve portfolio

As a benchmark portfolio, we chose to employ the introduced naïve benchmark prediction model (see Section 3.2). This framework is based on the *a priori* assumption that, in the optimization problem introduced in Equation (4.2), the expected returns vector $\boldsymbol{\mu}_t \in \mathbb{R}^a$ satisfies $\boldsymbol{\mu}_t = \mathbf{0}_a$ – expecting no returns in the following day, indirectly employing naïve model as the prediction engine. This framework is often referred to as the **Minimum Variance Portfolio** [26].

In this case, it is appropriate to analyze the performance of the set-up portfolio across different values of risk-aversion parameter λ_R. The dependence of CAGR on λ_R is depicted in Figure 4.1.

It is possible to observe that the CAGR converges to approximately 22.5% when $\lambda_R > 500$. However, the optimal value occurs at $\lambda_R = 300$, where the CAGR exceeds 23%.

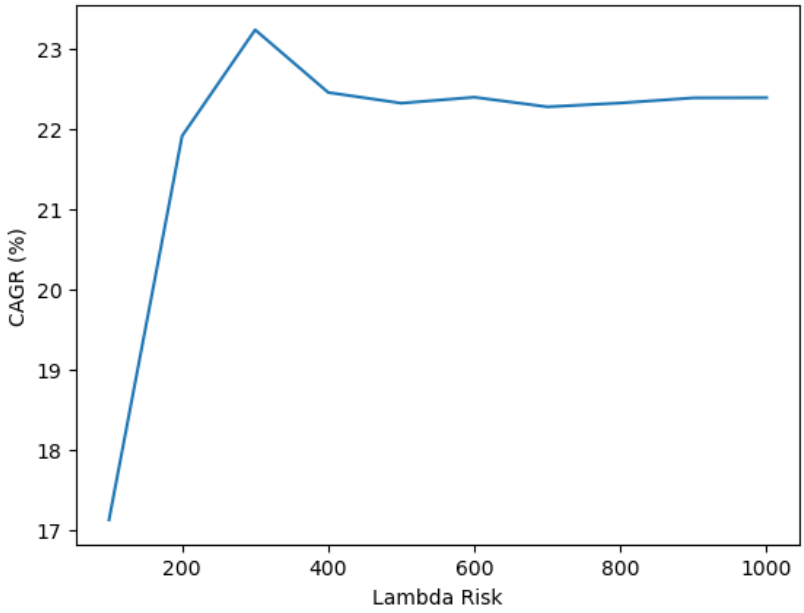

Figure 4.1: Dependence of CAGR on chosen λ_R value (Naïve model).

The dependence of AV on λ_R is depicted in Figure 4.2. A clear convergence is observed as λ_R increases, and the decline in AV for $\lambda_R > 1000$ remains negligible. This yields several possible management decisions. Either prioritize maximizing CAGR (even though with not optimal AV) or balance the two metrics to achieve approximately optimal results in both volatility and return.

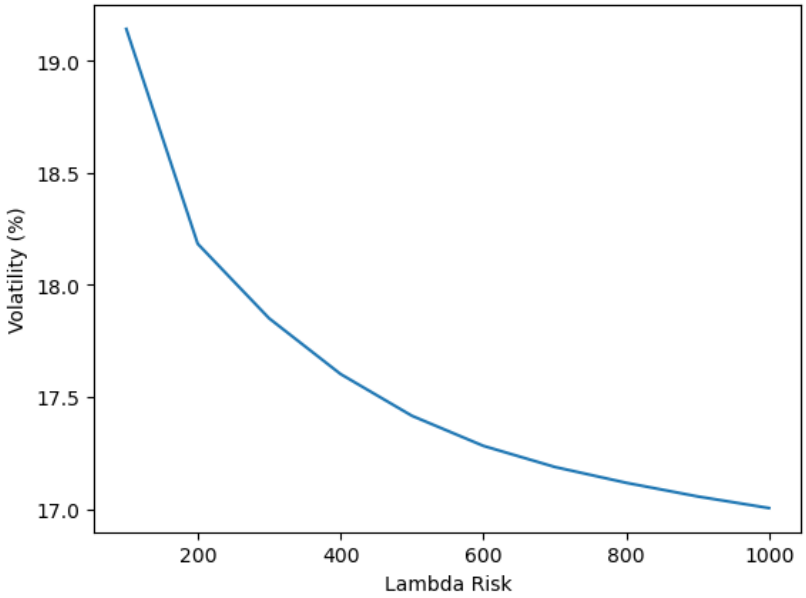

Figure 4.2: Dependence of AV on chosen λ_R value (Naïve model).

In this case, we chose to balance portfolio volatility and potential return by setting $\lambda_R = 1000$. The evaluation of this portfolio is presented in Table 4.1, alongside with performance of conventional indices available for investors.

Portfolio Type	**CAGR (%)**	**AV (%)**	**Sharpe ratio**
Naïve	22.38	17.01	1.32
S&P 500	7.69	21.75	0.35
NASDAQ	9.19	25.52	0.36

Table 4.1: Evaluation of the naïve portfolio, together with benchmark indices.

On average, the naïve portfolio allocated capital to **13.2** stock assets per day (out of a total 65).

A significant difference across all analyzed metrics between the naïve portfolio and the indices can be observed, with the naïve portfolio clearly outperforming both indices in every evaluation metric. This result is quite optimistic, even when accounting the transaction costs. This fact might root in segment focus of the portfolio on IT segment, which had experienced major surge in the recent years.

Lastly, to compare the portfolios, we simulate investment growth. Suppose an initial investment of 100 $ in September 2017. The development of this investment over the training-validation interval is shown in Figure 4.3

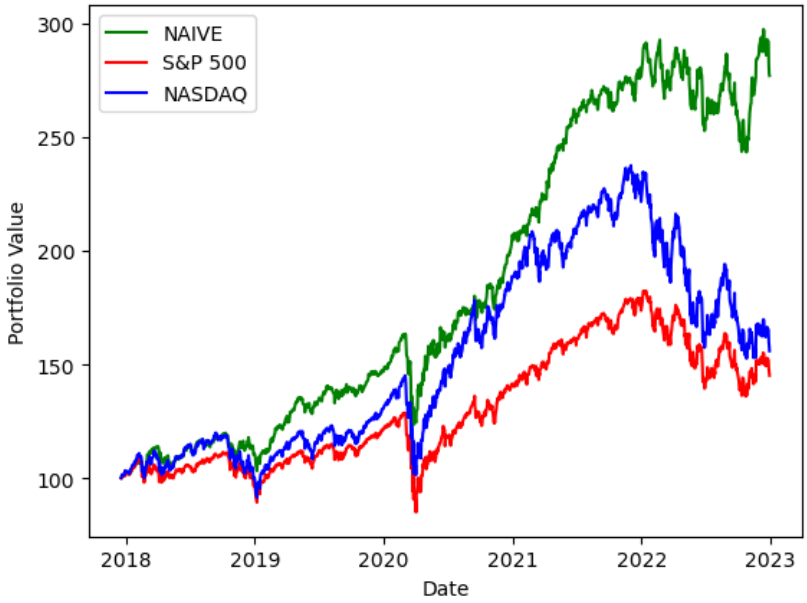

Figure 4.3: Investment development in the training-validation interval (Naïve portfolio).

4.3.2 Overview of risk-aversion (Lasso)

In Figure 4.4, the dependence of CAGR on the value of λ_R for the Lasso model is shown. Similar to the case shown in Figure 4.1, a direct relationship between CAGR and λ_R can be observed.

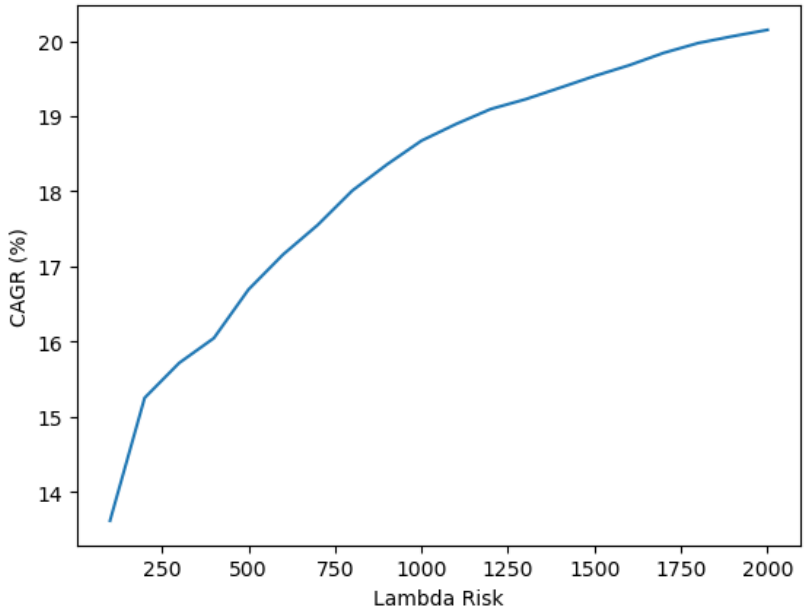

Figure 4.4: Dependence of CAGR on chosen λ_R value (Lasso model).

Furthermore, dependence of AV on the chosen value of λ_R is shown in Figure 4.5.

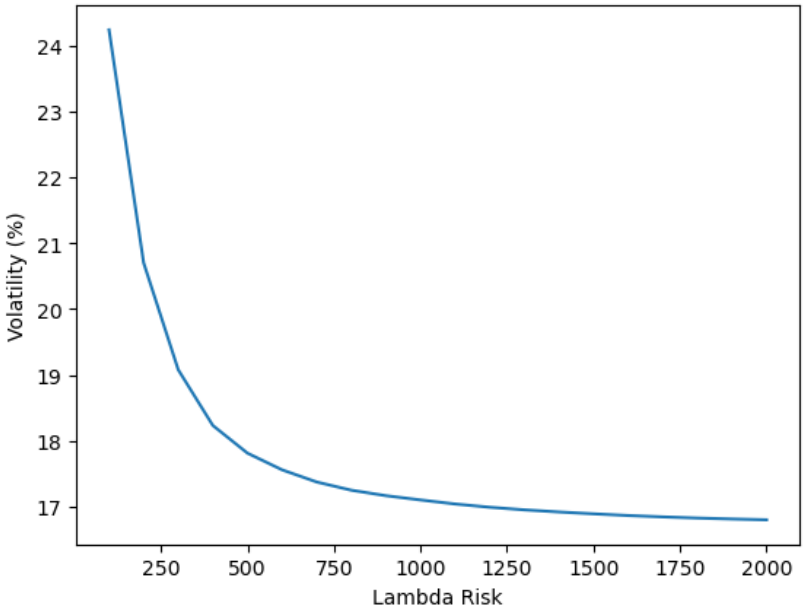

Figure 4.5: Dependence of AV on chosen λ_R value (Lasso model).

In terms of volatility management, this behaviour fully aligns with the role of λ_R in portfolio optimization. By making portfolio more conservative, overall volatility and risk decrease. Interestingly, AV appears to converge to approximately 17%. By increasing the value of λ_R over 1000, decline in AV is negligible.

Increasing the value of λ_R beyond 2000 yields only minimal gains in CAGR, on the order of $10^{-2} - 10^{-1}$%. Considering that further increases do not necessarily improve risk management, we set $\lambda_R = 2000$.

The resulting portfolio achieves a CAGR **20.15**%, an AV **16.8**% and a Sharpe ratio **1.2**. On average, capital was allocated to **11.1** stock per day (out of a total 65). Although the Lasso portfolio was unable to outperform naïve in terms of CAGR, it achieved slightly lower volatility, with AV reduced by approximately 0.2% in absolute terms.

To complete the evaluation, simulation of 100 $ investment is visualized in Figure 4.6

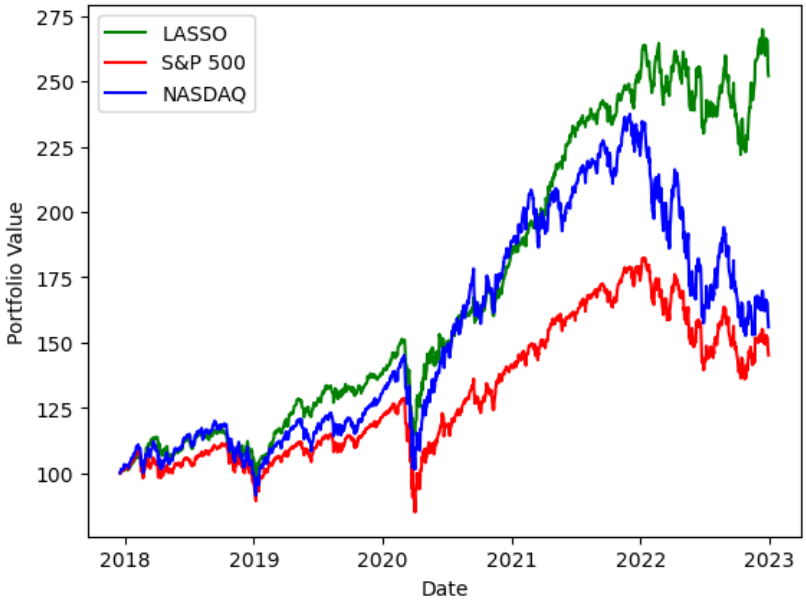

Figure 4.6: Evaluation of the Lasso portfolio, together with benchmark indices.

Compared to Figure 4.3, worse performance can be observed throughout 2021 and 2022. Interestingly, during the COVID-19 period (between 2021 and 2022), major indices declined; however, the proposed portfolio remained relatively stable. This suggests that the proposed portfolio might perform comparatively well during a bear market—a phase in which the overall market generally experiences negative returns.

4.3.3 Further tuning

At this point, an interesting observation occurred. Comparing Figures 4.4 and 4.5, it is possible to observe the same monotonic behaviour of both CAGR and AV with respect to the parameter λ_R across all employed prediction models. Even though, the actual optimal values for λ_R vary across models, the overall dependence overview stays the same. With this, it is appropriate to formally state the methodology used for selecting the optimal value of λ_R. By analyzing performance of portfolios on the given interval, it is possible (in all models) to identify a threshold value of λ_R beyond which improvement in both CAGR and AV becomes negligible. For all models, value of λ_R was selected at this threshold.

For this reason, we decided not to include dedicated tuning sections for each model, as the methodology and the process of tuning is identical across all considered models. Instead, this section summarizes the resulting optimal parameter settings for each particular model and the overall evaluation of the corresponding portfolios.

An overview of portfolio model settings is presented in Table 4.2. Note that for both Lasso and Ridge, the prediction model corresponds to the General model setting.

Portfolio Type	λ_R
ARIMA	1000
Ridge	1200
Lasso	2000

Table 4.2: Full overview of portfolio models tuned settings.

Subsequently, the performance of all tuned models can be analyzed together. An overview of the evaluation results is in Table 4.3.

Portfolio Type	**CAGR (%)**	**AV (%)**	**Sharpe ratio**	**Avg. # Stocks**
Naïve	22.38	17.01	1.32	13.2
ARIMA	22.25	16.94	1.31	11.9
Ridge	21.63	16.84	1.28	12.14
Lasso	19.87	16.77	1.18	11.1
S&P 500	7.69	21.75	0.35	–
NASDAQ	9.19	25.52	0.36	–

Table 4.3: Evaluation of all analyzed portfolio settings, together with benchmark indices (Training-validation interval).

There is clear evidence of the dominance of model-backed portfolios over the benchmark indices in all three metrics. In terms of CAGR, the best-performing portfolio was ARIMA. The ranking of portfolios by CAGR fully aligns with the ranking of prediction models by dominance (see Table 3.3). Interestingly, the best performing prediction model (in terms of average MAPE) did not perform the best in terms of CAGR. Furthermore, the behaviour of the AV metric is noteworthy. The lowest AV among model-backed portfolios was observed for the Ridge model, while the highest in Naïve model. This suggests that, in this case, employing predictions into the MVP framework generally reduces volatility compared to a portfolio that uses no next-day predictions.

Concluding the evaluation, model-backed portfolios substantially overperformed both indices across all metrics. This result may root from the performance of these models during the COVID-19 period, where major indices declined sharply, while the model-backed portfolios remained relatively stable. This suggests robust performance of all model-backed portfolios during crisis periods.

However, the Naïve portfolio achieved the highest CAGR, although not the lowest AV. This suggests that non-naïve models can manage market volatility more effectively than the naïve min-variance portfolio.

Finally, the overall performance of all considered portfolios is illustrated through the simulation of 100 $ investment over the training-validation interval, as depicted in Figure 4.7.

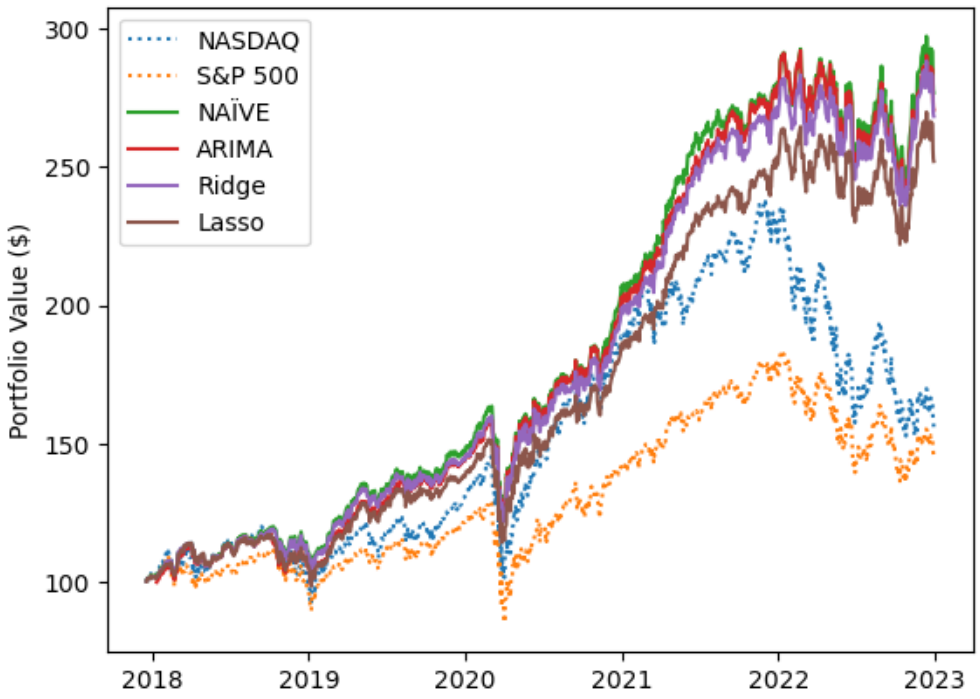

Figure 4.7: Evaluation of the portfolios, together with benchmark indices (Training-validation interval).

A fairly similar investment development can be observed for the Naïve, ARIMA and Ridge portfolios. The Lasso portfolio, however, experienced noticeably weaker performance in terms of return.

4.4 Evaluation on the testing interval

Eventually, the obtained portfolios were evaluated on the testing interval (January 2023 until January 2026). The evaluation results are presented in Table 4.4.

Portfolio Type	**CAGR (%)**	**AV (%)**	**Sharpe ratio**	**Avg. # Stocks**
Naïve	16.48	25.20	0.65	14.1
ARIMA	15.67	25.48	0.62	13.23
Ridge	15.30	25.24	0.61	13.1
Lasso	17.07	25.26	0.68	12.01
S&P 500	8.35	23.1	0.36	
NASDAQ	18.51	25.15	0.74	

Table 4.4: Evaluation of all analyzed portfolio settings, together with benchmark indices (Testing interval).

Quite the opposite behaviour is observed, compared to the training-validation inter-

val. A significant difference in CAGR is seen between the benchmark indices S&P 500 and NASDAQ. This is likely due to the so called **AI Boom**, during this period. In this interval, the technology and AI segments experienced a major surge in capitalization, resulting in historically above-average results [8]. This is reflected in major gains in tech-heavy NASDAQ index; however, the broader S&P 500 did not achieve such high returns.

Thanks to this factor, even our tech-heavy portfolios achieved relatively better results compared to the overall US market. However, these portfolios could not match the CAGR and AV of the NASDAQ. On the other hand, in contrast to the training-validation interval evaluation, particular model-backed (Lasso) portfolio overperformed the Naïve portfolio in terms of CAGR here.

The most interesting observation concerns the ranking of models based on CAGR. During the training-validation interval, models with the highest dominance achieved the best results. However, in the testing interval, the order changed, with the Lasso portfolio achieving the highest CAGR.

In conclusion, during the testing interval, certain model-backed portfolio was performing better in terms of CAGR than the Naïve portfolio; however, all the models were beat by the NASDAQ in both CAGR and AV. As a final evaluation, the overview of 100 $ investment development over time is depicted in Figure 4.8.

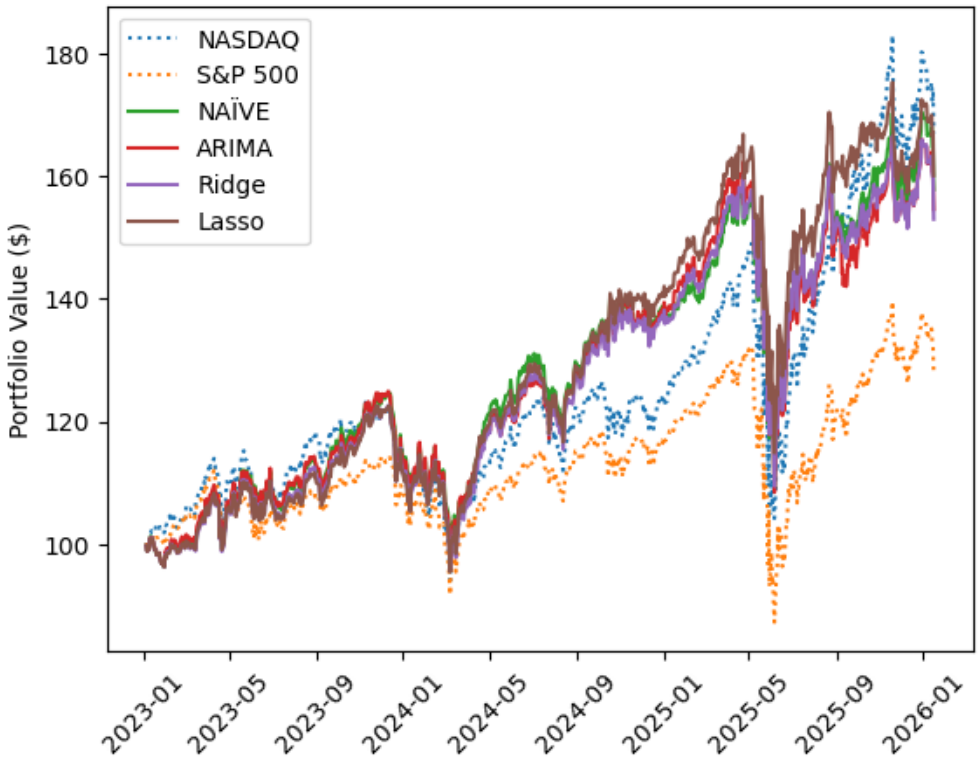

Figure 4.8: Evaluation of the portfolios, together with benchmark indices (Test interval).

Figure 4.8 shows a different development pattern, as there is no significant difference between benchmark indices and model-backed portfolios. Even so, NASDAQ outperformed all portfolios in terms of total absolute return. However, during the period between 2024 and 2025, model-backed portfolios exceeded the performance of both benchmarks.

What is more is the enormous dip during April and May 2025 (president Trump imposed first round of tarrifs) – in this period model-backed portfolios experienced similar decline compared to the benchmark indices. On the other hand, in terms of the total value of portfolio, all model-backed portfolios outperformed all of the indices.

4.5 Portfolio frequency analysis

As shown in Tables 4.3 and 4.4, we analyzed the number of stocks in portfolio per day. A deeper analysis of this data reveals some preference of particular models to certain stocks. In this section, the reported fractions of days indicate the proportion of days within interval during which a given stock was included in portfolio.

Analyzing training-validation interval, the Ridge model included all of the IT stocks in the portfolio for at least 100 trading days. In contrast, all other models completely excluded the stocks **Lam Research** (LRCX) and **Teradyne** (TER) from their portfolios.

On the other hand, all models showed preferences for various other stocks. However, the **IBM** stock was the most preferred among all; being included, in all of the the portfolios for more than 900 days in total (more than 66 % of days). Other frequently preferred stocks included **Motorola Solutions** (MSI), **Tyler Technologies** (TYL) or **GEN Technology** (GEN), each included on more than 50 % of days.

Among the least preferred stocks, apart from the LRCX and TER, the models rarely included **Palo Alto Networks** (PANW) (at most 8% days), **Microchip Technology** (MCHP), or **ON Semiconductor** (ON) (both at most 10 % days).

The preference for certain stocks remained largely consistent, with only slight shifts in ranking during the testing interval. However, the most included stock changed – during the testing interval, the most included stock was **Verisign** (VRSN) (over 92 % of days), followed by mentioned **Motorola Solutions** (approximately 85 % of days) and (the most preferred stock on training-validation interval) **IBM** (72 % of days). Interesting is the fraction of trading days, in which most preferred stocks were included in training-validation and test intervals. On test interval, frameworks included these stocks vastly more often than on the training-validation interval.

On the other hand, all portfolios completely excluded few stocks. For instance,

Oracle Corporation (on training-validation interval included approximately in 25 % of days) and **Lam Research** (on training-validation interval excluded as well).

An interesting visualization in Figure 4.9 highlights the periods during which the most and least frequently used stocks were included in the Ridge model portfolio (training-validation interval). The Ridge model was chosen for this illustration because it did not completely exclude any of the stocks.

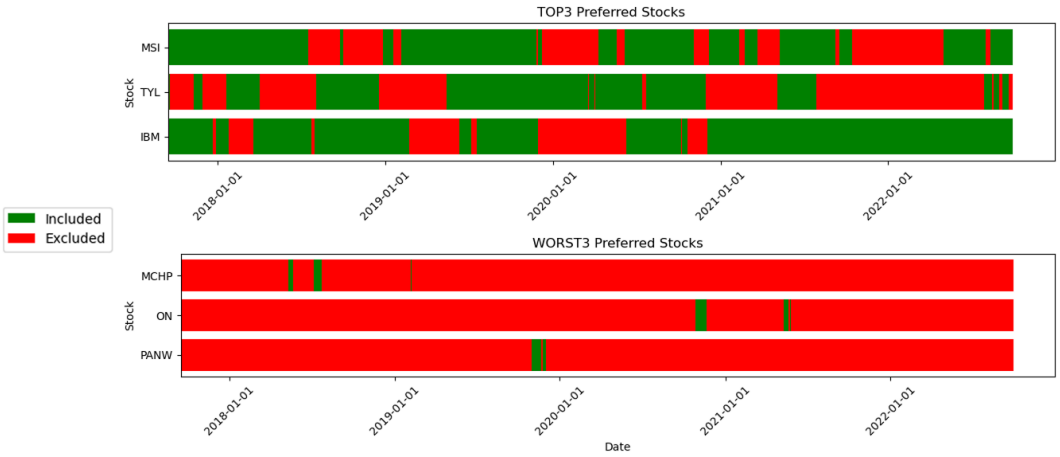

Figure 4.9: Overview of use of certain stocks in Ridge model (Training-validation interval).

Figure 4.10 overviews the actual proportion of capital being allocated to the most used stocks. These three stocks made up in average **26.36** % of the portfolio.

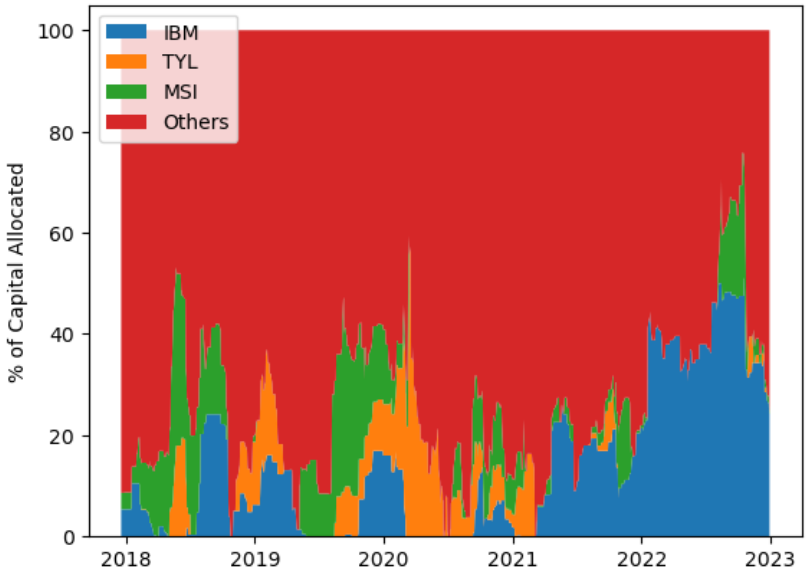

Figure 4.10: Overview of capital use of certain stocks in Ridge model (Training-validation interval).

What is more, the three most included stocks made up the highest proportion of

portfolio in average as well. In the respective order, IBM **13.93** %, MSI **7.53** % and TYL **4.91** % in average.

Conclusion

In the thesis, we focused on two main goals. Firstly, to design and build a machine-learning-based prediction model capable of predicting the next-day price of individual stocks from the chosen market segment. Secondly, we focused on employing the acquired models in subsequent portfolio optimization and strategy building using the MVP framework.

Chapter 1 introduced the fundamental concepts of ML and modeling to the reader. The chapter presented the foundations of all algorithms and frameworks used in the thesis and briefly overviewed the concepts of ML theory, which are frequently referred to in the thesis.

A specific training, validation, and testing splitting concept was adopted in the thesis. Taking into account the day-to-day prediction methodology, we adopted a rolling window, where the latest available predictor data (acquired from the constantly long interval) are used to predict the next-day price of a particular stock. On top of that, we used a unified set of predictors, consisting largely of the market data. Chapter 2 introduced the concepts of the rolling window, data preparation and the set of predictors used in the thesis.

Subsequently, it was possible to further evaluate the adopted ML frameworks and optimize their settings in order to achieve the best prediction results. The results showed that stocks in the IT segment, in the particular time interval, were difficult to predict. Naïve prediction showed a great potential and justified that stock price development aligns more with the random walk [7].

The introduced metrics of dominance and MAPE showed that vast majority of employed models were outperformed by the Naïve prediction in terms of prediction accuracy. Only one framework (ARIMA model) outperformed the Naïve model in terms of dominance, which showed that time series models might be more capable of generalizing the pattern than conventional ML models. This result is often not stated in a number of studies (e.g., [27], [32], [35]), which might lead to an overestimation of the models' practical utility.

Subsequent analyses of the application of the models to the portfolio optimization were introduced in Chapter 4. Evaluation results suggested that even though prediction of most of the models were inferior to the Naïve benchmark, the performance of

portfolios, backed by models, was above-average compared to the conventional stock indices. Moreover, the portfolio backed by the Naïve model delivered comparable results to other non-naïve models.

On the other hand, quite a counterintuitive fact arose in the analyses. While during the period between 2017 and 2022, models with the lowest dominance delivered the best-performing portfolios, during the period between 2023 and 2026 the opposite emerged. The portfolio backed by the Lasso model (which was evaluated as the least dominant model) was the best performer (on the testing set) in terms of average annual return across all the considered models. Even so, the NASDAQ index outperformed the portfolios, which is opposite to the previous data sample as well.

Comparing both analyzed intervals, the results of the research suggest that the proposed portfolio models (accompanied with the prediction modeling) tend to be less volatile and profitable during periods of neither a bull nor a bear market than Naïve portfolio or conventional indices. On the other hand, during a strong bull market (in this case, the technology segment), strategy performed worse than the NASDAQ index.

Aligning with Palomar [26], the actual predictions tend to be less important for the portfolio optimization than the actual covariance of returns – realized volatility of the stocks.

Apart from the compared studies (e.g., [18], [35], [36]), we researched possibilities of applying such predictions for portfolio management. The referred studies focused solely on designing the prediction pipeline and prediction evaluation. Connecting this methodology with portfolio optimization increases the application value of the research done in the thesis and brings an unconventional point of view regarding the model's evaluation. Compared to the studies implementing ML together with portfolio optimization (e.g., [4], [10], [12], [21]), we implemented detailed evaluation on recent market data and focused mainly on one coherent set of stocks.

Used ML algorithms cannot comprehend the time-flow dependency properly [5]. However, by using the ARIMA model we addressed the issue. Even though time series models are used in the research quite often, simpler frameworks are used more, even with inferior performance [29]. This suggested a suitable alternative to the conventional ML frameworks.

As stated earlier, prediction Naïve benchmark showed competitive results, in comparison with the other models. This shows that there exists a less computationally heavy prediction alternative, capable of outperforming advanced ML frameworks. Some studies tend to compare models between each other [29] or omit the comparison at all (e.g., [32], [36]). The thesis shows potential worse performance of the models compared to the computationally inexpensive approaches.

Quantitative and qualitative comparison with the existing studies is limited. Direct

comparison cannot be done, as there is no widely known study conducting similar rolling-window prediction on the whole IT market segment. However, several implicit conclusions can be done. Sezer et al. [29] suggest that many studies show various best performing frameworks. This suggests that particular models might perform diversely on various assets across the history.

Moreover, the ML workflow of applying models out-of-sample tends to be maintained [18, 29, 36]. However, majority of the examined studies used static time series splitting – static time points break the whole data segment into particular ML sets. Minority of studies implemented rolling window methodology [29].

Such an analysis of any coherent group of stocks could be performed using the designed pipeline. For future developments, it would be beneficial to conduct such research for the whole US market, which might create a trustworthy foundations for informed decision-making in the market. Lastly, by researching and implementing more sophisticated portfolio optimization frameworks, the overall performance of the investment might improve.

Apart from that, there is possibility of implementing more advanced frameworks – neural networks. Some studies suggest more reliable prediction results than conventional ML algorithms [29]. However, we decided not to include neural networks in this research, as the data preparation implemented does not align with the requirements of other used frameworks. Altering the preparation process just for the purposes of neural networks would bias the analyses.

Lastly, regarding the goals of the thesis, all of the targeted aims were fully accomplished.

Bibliography

[1] Ran Aroussi. yfinance: Yahoo! Finance market data downloader. Available at: `https://github.com/ranaroussi/yfinance`, 2025. Accessed: 2025-09-29.

[2] Bank for International Settlements. OTC foreign exchange turnover in April 2025. Statistical release, Bank for International Settlements, September 2025. Preliminary results; Final data released December 2025.

[3] Dave Bergmann. What is machine learning? Available at: `https://www.ibm.com/think/topics/machine-learning`, 2025. Accessed: 2025-02-01.

[4] Apichat Chaweewanchon and Rujira Chaysiri. Markowitz mean-variance portfolio optimization with predictive stock selection using machine learning. *International Journal of Financial Studies*, 10(3), 2022.

[5] Francis X. Diebold. *Elements of Forecasting*. Thomson South-Western, 4th edition, 2006.

[6] Nikita Chhabra et al. 2024 Dividend Trends: A Global Overview. Visual Capitalist Elements, September 2024. Accessed: 2025-12-04.

[7] Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *Journal of Finance*, 25(2):383–417, 1970.

[8] Daniel J. Graeber. S&P 500 in formal bull market territory, up 20% from October levels. United Press International (UPI), June 9 2023. Accessed: 2026-02-27.

[9] Jakub Grudniewicz and Rafal Ślepaczuk. Application of machine learning in algorithmic investment strategies on global stock markets. *Research in International Business and Finance*, 66:102045, 2023.

[10] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 05 2020.

[11] Christian M. Hafner and Linqi Wang. Dynamic portfolio selection with sector-specific regularization. *Econometrics and Statistics*, 32:17–33, 2024.

[12] J. B. Heaton, Nicholas G. Polson, and J. H. Witte. Deep learning in finance. *CoRR*, abs/1602.06561, 2016.

[13] Yuping Huang. Research on the Google Stock Price Prediction Based on SVR, Random Forest, and KNN Models. *Highlights in Business, Economics and Management*, 24:1054–1058, 2024.

[14] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning*. Springer Texts in Statistics. Springer Nature, 2023.

[15] Xinyue Ji. Comparison of portfolio optimizations under markowitz model in technology sector and financial services sector. *Highlights in Business, Economics and Management*, 24:1194–1202, Jan. 2024.

[16] Will Kenton. S&P 500 Index: What It's for and Why It's Important in Investing, November 20 2025. Accessed: 2025-12-03.

[17] Luckyson Khaidem, Snehanshu Saha, and Sudeepa R. Dey. Predicting the direction of stock market prices using random forest. *Expert Systems with Applications*, 83:86–99, 2017.

[18] Christopher Krauss, Xaun Do, and Nicolas Huck. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2):496–502, 2017.

[19] B. Rajesh Kumar. Stock markets, derivative markets, and foreign exchange markets. In *Strategies of Banks and Other Financial Institutions: Theories and Cases*, chapter 5, pages 121–141. Academic Press, San Diego, 2014.

[20] Ronald R. Kumar, Peter J. Stauvermann, and Hang T. T. Vu. The Relationship between Yield Curve and Economic Activity: An Analysis of G7 Countries. *Journal of Risk and Financial Management*, 14(12):574, 2021.

[21] Yilin Ma, Ruizhu Han, and Weizhong Wang. Portfolio optimization with return prediction using deep learning and machine learning. *Expert Systems with Applications*, 165:113973, 2021.

[22] Pablo Mantilla and Sebastián Dormido-Canto. A novel feature engineering approach for high-frequency financial data. *Engineering Applications of Artificial Intelligence*, 125:106394, 2023.

[23] Akito Matsumoto, Andrea Pescatori, and Xueliang Wang. Commodity prices and global economic activity. *Japan and the World Economy*, 66:101117, 2023.

[24] Charlotte Morabito. The S&P 500 is more concentrated with AI than ever. Here's how to manage your risk, October 22 2025. Accessed: 2025-12-03.

[25] Jasper Osita. Why Gold Remains the Ultimate Security in a Shifting World. ACY Securities, April 22 2025. Accessed: 2025-11-27.

[26] Daniel P. Palomar. *Portfolio Optimization: Theory and Application*. Cambridge University Press, 2025.

[27] Jigar Patel, Sahil Shah, Priyank Thakkar, and K Kotecha. Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1):259–268, 2015.

[28] Sakina and Muhammad A. Khan. Stock Price Prediction using Neural Networks. *Pakistan Journal of Law, Analysis and Wisdom*, 3:123–135, 2024.

[29] Omer B. Sezer, Ugur M. Gudelek, and Ahmet M. Ozbayoglu. Financial Time Series Forecasting with Deep Learning : A Systematic Literature Review: 2005-2019. *Applied Soft Computing*, 90:106181, 2020.

[30] Steven S. Skiena. *The Data Science Design Manual*. Texts in Computer Science. Springer Nature, 2017.

[31] Taylor G. Smith. pmdarima. Available at: `https://github.com/alkaline-ml/pmdarima`, 2026. Accessed: 2026-02-23.

[32] Mehar Vijh, Deeksha Chandola, Vinay Anand Tikkiwal, and Arun Kumar. Stock closing price prediction using machine learning techniques. *Procedia Computer Science*, 167:599–606, 2020. International Conference on Computational Intelligence and Data Science.

[33] Qi Wang, Chang Xu, and Tieyan Zhou. Stock price prediction based on multiple linear regression. *BCP Business & Management*, 36:48–54, 2023.

[34] Yahoo Finance. Yahoo Finance. Available at: `https://finance.yahoo.com/`, 2025. Accessed: 2025-09-29.

[35] Leying Yin, Xinyu Zhang, and Xiwen Xiong. Application and Effectiveness of KNN Model in Stock Market Prediction. In *Proceedings of the 3rd International Conference on Financial Technology and Business Analysis*, pages 129–133, 2025.

[36] Yuqing Zhao. Stock Price Prediction Based on Linear Regression. In *Proceedings of the 4th International Conference on Computing Innovation and Applied Physics*, EAI/Springer Innovations in Communication and Computing, pages 85–90. EAI, 2025.

Appendix A: Source code of the model

The Appendix of this thesis presents all the source codes used in all of the analyses. All the source codes and results are freely available as a Git repository at `https://github.com/sugy-w/Bachelor-Thesis`.

Please note, that all the contents of this repository is being shared only for academic purposes and its modifying, redistribution or commercial use is possible according to the enclosed **license** only.

To start, see the enclosed `README`, which summarized functionalities of the source code and possibilities of replication, accordingly to the thesis.